CentraleSupélec

université PARIS-SACLAY

Big Data Management & Analytics Master

NFTM for Computer Vision: Adaptive Inpainting and Denoising

Kristóf Balázs
Stefanos Kypritidis
Nicola Mambelli

Github Link

Advisor: Nacera Seghouani - CentraleSupélec nacera.seghouani@centralesupelec.fr
Advisor: Akash Malhotra - Amadeus akash.malhotra@centralesupelec.fr

Contents

1 Introduction **1**

2 Background **3**
2.1 NFTM Based Image Inpainting . . . 3
2.1.1 Iterative Correction via Controller . . . 4
2.1.2 Damping Factor β . . . 4
2.1.3 Guard Mechanism . . . 5
2.1.4 Masking and Input Corruption . . . 6
2.1.5 Energy and Regularisation (MSE, TV-L1, Contractive Term) . . . 6
2.1.6 Rollout Curriculum and Step Scheduling . . . 7
2.1.7 Controller Architectures: Dense vs. U-Net . . . 7
2.1.8 Step Clipping and Gating . . . 7
2.1.9 Metrics and Evaluation Protocol . . . 8
2.1.10 Flags Reference . . . 8
2.2 Metrics . . . 9
2.2.1 PSNR . . . 9
2.2.2 SSIM . . . 9
2.2.3 LPIPS . . . 10
2.2.4 FID . . . 10
2.2.5 KID . . . 11

3 Relative works **12**
3.1 Existing Approaches . . . 12
3.1.1 TV-L1 (Total Variation – L1) . . . 12
3.1.2 Single-Step U-Net . . . 12
3.1.3 Diffusion Models . . . 13

4 Experimental Setup **14**
4.1 Datasets . . . 14
4.1.1 Cifar . . . 14
4.1.2 CelebAHQ . . . 14
4.1.3 SIDD . . . 15
4.2 Benchmark datasets . . . 16
4.2.1 CBSD68 . . . 16
4.2.2 Set12 . . . 16
4.2.3 Noise Types and Corruption Protocol . . . 17
4.3 Diffusion Baseline Implementation . . . 19
4.3.1 DDPM . . . 19
4.3.2 Architectures . . . 20
4.3.3 Results . . . 21
4.4 Experiment Configuration . . . 21

4.5 Experiment Environment Setup . . . 22
4.6 Experiment Monitoring . . . 22

5 Experiments and Evaluation 23
5.1 Initial Setup . . . 23
5.1.1 Baseline configuration . . . 23
5.1.2 Controller architectures . . . 24
5.1.3 Additional baselines: one-shot U-Net and TV-L1 . . . 25
5.1.4 One-click experiment orchestration . . . 26
5.2 Successful and Failed Additions . . . 26
5.2.1 Coarse-to-fine Inpainting (Pyramid) . . . 26
5.2.2 Step-Loss . . . 28
5.2.3 Residual Connections . . . 28
5.2.4 Batch Normalisation . . . 30
5.2.5 RNN Controller . . . 31
5.2.6 SIDD Real Camera Noise Test . . . 32
5.2.7 Other additions . . . 36
5.3 Experiments not Relevant for the Best Configuration . . . 36
5.3.1 K_train and K_eval . . . 36
5.3.2 Robustness to Different Noise Distributions in Inpainting . . . 37
5.4 Analysis of the Most Influential Components . . . 39
5.4.1 β and Rollout Types . . . 40
5.4.2 `corr_clip` . . . 41
5.4.3 `K_train` and `K_eval` . . . 42
5.4.4 Learning Rate α . . . 43
5.4.5 Guard and Backtracking . . . 45
5.4.6 Regularization Through Flags (`tv_weight`, `contract_w`, `weight_decay`) . . . 45
5.4.7 Best Configuration Inpainting . . . 46
5.4.8 Best Configuration Denoising . . . 48
5.4.9 Controlling the Task Behavior Through Flags . . . 51
5.5 Main Findings from Experiments . . . 52
5.5.1 Differences between baselines - Inpainting . . . 52
5.5.2 Summary of Best Achieved Results . . . 52

6 Conclusions and Future Work 54
6.1 Conclusions . . . 54
6.2 Limitations . . . 55
6.3 Future Work . . . 55

A Appendix 57

Bibliography 70

Chapter 1

Introduction

Image inpainting is the task of reconstructing missing or corrupted regions of an image in a way that remains consistent with the visible context and looks plausible to a human observer. It appears in many practical settings: removing occlusions, restoring damaged photographs, filling sensor dropouts, and enabling downstream vision pipelines to operate on incomplete inputs. This is fundamentally challenging because inpainting is inherently ill-posed: many completions can match the same observed pixels, and the "right" solution depends on both low-level texture continuity and high-level semantic structure. Traditional approaches often struggle to balance these requirements: some preserve edges and smoothness but fail semantically, while others hallucinate plausible content but break local fidelity.



Figure 1.1: General example of image inpainting (adapted from [1]).

Image denoising targets a closely related but distinct restoration setting. Instead of missing pixels, the full image is observed but contaminated by noise introduced by sensing and acquisition (e.g., low-light shot noise, sensor read noise, and ISP artifacts). The goal is to remove noise while preserving fine detail and avoiding oversmoothing or texture wash-out. As with inpainting, denoising is ill-posed: multiple clean images can explain the same noisy observation, and the difficulty increases when noise is signal-dependent or spatially varying, as in real camera pipelines. Methods that are too conservative leave residual noise; methods that are too aggressive erase structure and introduce unnatural artifacts.

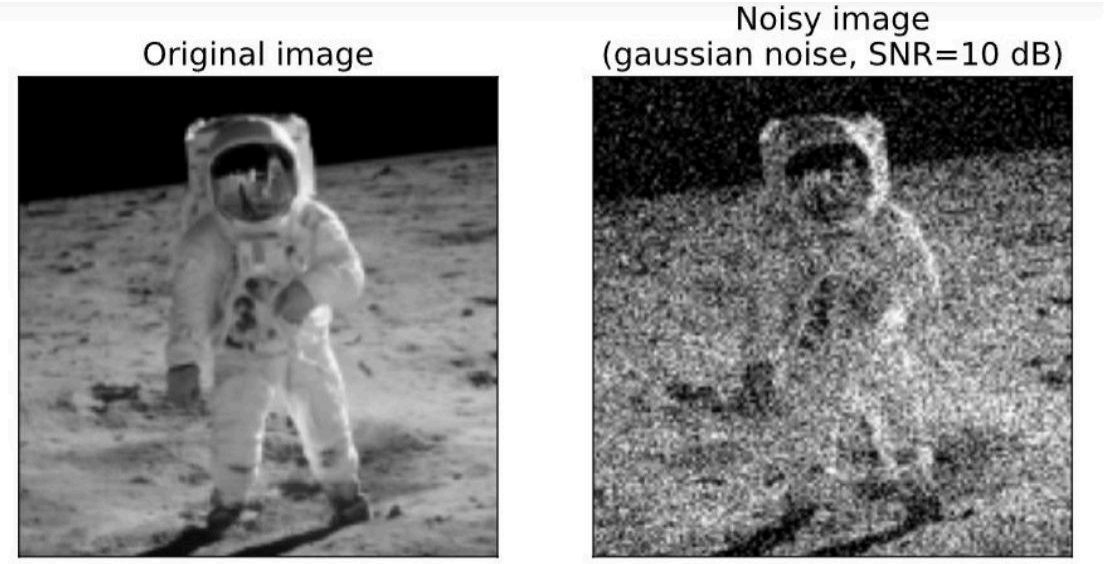


Figure 1.2: Example of image denoising (adapted from [2]).

In this work, we frame both tasks as iterative reconstruction under a known corruption model. For inpainting, the corruption is a binary mask specifying which pixels are observed; for denoising, the corruption corresponds to additive perturbations (and, when available, real-noise observations) affecting all pixels. In both cases, the objective is to progressively refine an estimate over multiple steps, using feedback from the current reconstruction rather than relying on a single-shot prediction.

A key design goal is deliberate simplicity: we prioritize a minimal, modular pipeline with limited architectural and procedural complexity, aiming for robustness and clarity over increasingly elaborate mechanisms. While the main focus is inpainting, we also evaluate the same iterative formulation on denoising to test whether the approach transfers beyond masked reconstruction.

Chapter 2

Background

2.1 NFTM Based Image Inpainting

The Neural Field Turing Machine (NFTM) is introduced as a differentiable computational framework for operating on spatial fields through local, iterative updates, bridging ideas from neural fields, memory-augmented computation, and dynamical systems [3]. Rather than treating data as sequences of tokens or discrete memory slots, NFTM represents state as a field defined over a continuous domain and evolves this state over discrete timesteps. Computation is mediated by a neural controller coupled with explicit read/write operations over local neighborhoods (support regions) in the field. This locality constraint induces linear scaling with the number of spatial sites for a fixed support radius, and the paper positions NFTM as a unifying abstraction encompassing neural cellular automata, neural Turing machines (in a spatially grounded form), and deterministic iterative refinement procedures.

In the context of image inpainting, the task can be interpreted as perceptual inference via field evolution: the image is treated as the spatial state, and missing-content completion is achieved by iterating a learned update rule until the estimate becomes consistent with the observed context and plausible globally. Under this interpretation, inpainting is no longer framed as a single forward mapping from corrupted input to output, but as a controlled dynamical process that trades computation (number of rollout steps) for solution quality. The NFTM paper explicitly includes iterative image refinement (e.g., CIFAR-10 inpainting) as one of its representative instantiations, illustrating how the same architectural principle (local read/compute/write updates over a spatial state) can be applied to perceptual reconstruction problems alongside symbolic and physical domains.

Figure 2.1 represents one NFTM inpainting step in our current implementation. Most of the functionality depicted on this diagram is discussed in this Section below.

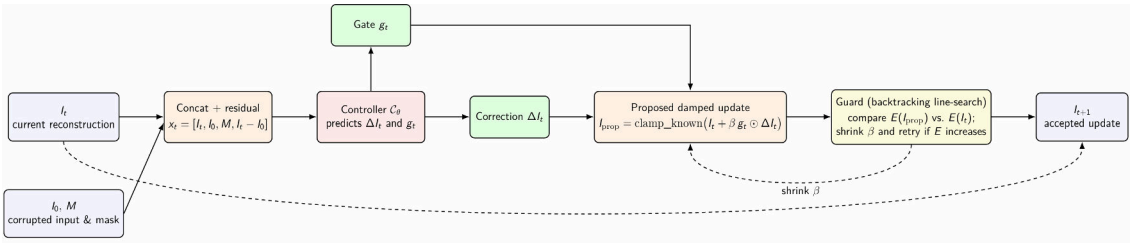


Figure 2.1: One NFTM inpainting step in our implementation.

2.1.1 Iterative Correction via Controller

The inpainting method is cast as an iterative correction process, where a parametric controller defines the update direction at each refinement step. Starting from a corrupted initialization I_0 that satisfies the known-pixel constraint, the estimate is updated over K steps by repeatedly querying a controller g_θ that operates on the current state and the mask:

$$(dI_t,\ g_t) = g_\theta(I_t, M), \qquad I_{t+1} = I_t + \beta\ \mathrm{clip}\big(g_t \odot dI_t,\ \pm c_t\big),$$

followed by clamping on the observed region. In this view, the controller is not a single-shot generator of missing content; instead, it acts as a learned local update rule that progressively improves the reconstruction. The iterative formulation is particularly useful for inpainting because it allows repeated re-evaluation of the current estimate: early steps can focus on coarse structure and boundary consistency, while later steps can refine textures and remove residual artifacts.

The controller serves two complementary purposes. First, it provides a flexible mechanism for conditioning on both the current image estimate and the observation mask, enabling the update to be spatially adaptive and mask-aware. Second, it defines a modular interface between the rollout engine (step scheduling, damping, clipping, optional guards) and the learned component. This separation makes the controller swappable: the remainder of the inpainting pipeline is agnostic to the internal architecture as long as the controller maps (I_t, M) to a correction field and a gate. As a result, controller capacity can be increased or decreased without changing the optimization and evaluation protocol, enabling controlled comparisons between architectures under identical iterative dynamics.

Two controller families are currently supported. The first is a lightweight convolutional controller (`TinyController`) that uses a shallow conv stack to predict a bounded per-pixel correction (via tanh) and a per-pixel gate (via sigmoid). This variant is parameter-efficient and emphasizes the role of the iterative rollout as the primary source of expressive power. The second is a UNet-based controller (`UNetController`) that predicts the same outputs with a multi-scale encoder–decoder backbone, providing stronger spatial context and improved modeling of larger structures. Both controllers conform to the same input/output contract, ensuring that the rest of the algorithm remains unchanged.

Future work can extend this controller interface in several directions without modifying the rollout machinery. Architecture-wise, more expressive backbones could be introduced as controllers, for example, deeper UNets, attention-based modules, or diffusion-style denoisers.

2.1.2 Damping Factor β

In our NFTM update rule, the controller predicts a per-pixel correction dI and a gate g, and the image is updated as

$$I_{\text{new}} = I_{\text{old}} + \beta\, g \odot dI,$$

where $\beta > 0$ is a global damping factor that controls the effective step size of the update.

Intuitively, a large β allows the model to take aggressive steps toward the predicted image, which can accelerate convergence but also risks overshooting and creating instabilities or oscillations. A smaller β produces more conservative updates, leading to smoother but slower refinement.

In the current implementation, β plays two roles:

- **Base step size during training and evaluation.** We start from an initial value β_{start} and (optionally) anneal it across (with β_{anneal} steps per epoch) epochs up to a maximum $\beta_{\max}$, which lets the controller first learn stable local corrections before being trusted with larger steps. We can also add a $\beta_{\text{eval_bonus}}$, which adds extra β at evaluation time.
- **Dynamic damping inside the guard mechanism.** Detailed in the next Subsection 2.1.3.

At this stage, the damping factor is still an experimental component: we keep it in the pipeline, but later ablations will decide whether a fixed β (or even $\beta \equiv 1$ with tighter clipping) is good enough. Details on the experiments with β and its related flags are detailed in Chapter 5.

2.1.3 Guard Mechanism

To make the whole iterative correction process more stable, at each step we use a guard mechanism, which acts as a simple backtracking line-search on the inpainting energy. At a high level, the guard prevents the refinement process from taking steps that increase the energy, so it basically helps to avoid divergence when the controller proposes overly aggressive corrections.

To explain this better, let $E(I)$ denote the inpainting energy (MSE + TV-L1) of a candidate image I with respect to the ground truth:

$$E(I) \;=\; \underbrace{\|I - I_{\text{gt}}\|_2^2}_{\text{data term}} \;+\; \lambda_{\text{tv}}\,\text{TV}(I).$$

For each NFTM step we do the following:

1. Compute the current energy $E_0 = E(I)$.
2. Using the controller and current step size β, propose an updated image I_{prop}:

   $$I_{\text{prop}} = I + \beta\, g_\theta(I, M),$$

   where g_θ denotes the controller output (gated correction) and M is the known–pixel mask.
3. Evaluate the new energy $E_1 = E(I_{\text{prop}})$.
4. If $E_1 \leqslant E_0$, the step is accepted and we set $I \leftarrow I_{\text{prop}}$.
5. Otherwise, the step is considered too aggressive. We shrink the step size by a constant factor,

   $$\beta \leftarrow \beta \cdot \texttt{shrink},$$

   with $\texttt{shrink} \in (0, 1)$ (e.g. $\texttt{shrink} = 0.5$), and recompute a proposed image. This backtracking is repeated up to `max_backtracks` times.

6. If none of the tries give a lower energy, the step is rejected and we keep the previous I unchanged.

This is optional during training (it improves stability but adds extra forward passes), but it is the most useful at inference: it prevents the reconstruction from diverging when β is large or when the controller proposes corrections that increase the energy.

2.1.4 Masking and Input Corruption

Inpainting is formulated as reconstruction under a known binary mask $M \in \{0,1\}^{H\times W}$, where $M_{ij} = 1$ denotes observed pixels and $M_{ij} = 0$ denotes missing pixels. For each mini-batch, a new mask is sampled using `random_mask` with a missing fraction uniformly drawn in $[p_{\min}, p_{\max}]$ and optional structured occlusions (random blocks) controlled by `block_prob`. This yields a mixture of corruption regimes, including scattered missingness and contiguous holes, preventing reliance on a single mask pattern.

Given the ground-truth image I_{gt}, a corrupted initialization I_0 is generated by injecting Gaussian noise only on the missing region (standard deviation `noise_std`), while observed pixels remain consistent with the input evidence:

$$I_0 = \texttt{corrupt_images}(I_{\text{gt}}, M;\ \sigma = \texttt{noise_std}).$$

A hard constraint is then enforced via `clamp_known`, which explicitly overwrites the observed pixels in the current estimate with their ground-truth values. As a result, the rollout state always satisfies

$$I \odot M \ = \ I_{\text{gt}} \odot M,$$

meaning that updates are effectively restricted to the missing region. In practice, this constraint is applied at initialization and re-applied whenever the estimate is modified by subsequent processing steps, ensuring that the observed region never drifts away from the input evidence.

2.1.5 Energy and Regularisation (MSE, TV-L1, Contractive Term)

The guard mechanism introduced in the previous section relies on an explicit energy formulation to evaluate and control the behavior of the iterative refinement process.
This energy provides a scalar criterion that quantifies the quality of a candidate reconstruction by jointly accounting for fidelity to the ground truth and spatial regularity of the reconstructed image.
Formally, given a candidate image I and the corresponding ground-truth image I_{gt}, the inpainting energy is defined as

$$E(I) \ = \ \underbrace{\|I - I_{\text{gt}}\|_2^2}_{\text{data term}} \ + \ \underbrace{\lambda_{\text{tv}}\mathrm{TV}(I)}_{\text{regularization}} \ .$$

The data term measures the discrepancy between the reconstructed image and the ground truth, encouraging accurate recovery of missing content. The regularization term enforces spatial smoothness by penalizing large intensity variations between neighboring pixels, thereby discouraging abrupt artifacts and unstable local corrections.

By combining these two components, the energy captures both reconstruction accuracy and structural coherence.
Monitoring this energy during the iterative updates enables principled decisions about the acceptability of proposed corrections, and forms the foundation for the stabilization strategy implemented by the guard mechanism.

2.1.6 Rollout Curriculum and Step Scheduling

Training is performed over multi-step rollouts rather than single-shot prediction. The maximum rollout depth follows a curriculum that increases with the epoch index:

$$K_{\text{train}} = \min\big(K_{\text{target}},\, K_{\text{base}} + \texttt{epoch}\big),$$

with $K_{\text{base}} = 4$ and $K_{\text{target}} = \texttt{K_train}$. For each batch, an actual rollout length $K_{\text{curr}} \in \{1, \ldots, K_{\text{train}}\}$ is sampled; when `rollout_bias` is enabled, shorter rollouts are sampled with higher probability (geometric weighting), prioritizing stable short-horizon corrections in early training. During evaluation, the rollout length is fixed to `K_eval` to ensure consistent comparisons.

The update magnitude is controlled by the global damping factor β as mentioned already.

2.1.7 Controller Architectures: Dense vs. U-Net

We consider two alternative controller architectures with identical input–output interfaces: a dense convolutional controller and a compact U-Net-based controller. Both controllers take as input the concatenation of the current image estimate I_t and the binary mask M, resulting in a 4-channel input tensor, and produce three outputs: an RGB correction, a per-pixel gate, and a per-pixel log-variance term.

The dense controller is a deliberately minimal architecture. It consists of three sequential 3×3 convolutional layers with GELU activations and a fixed channel width. From the resulting feature representation, three independent convolutional heads are applied to predict a 3-channel RGB correction, a 1-channel gating map, and a 3-channel log-variance map. The RGB correction is passed through a tanh non-linearity, while the gate is squashed using a sigmoid function.

As a more expressive alternative, the U-Net controller adopts a compact encoder–decoder architecture with skip connections. It processes the same 4-channel input and produces a 7-channel output, which is split into an RGB correction, a gating signal, and a log-variance term. The hierarchical structure of the U-Net enables multi-scale feature aggregation, allowing the controller to model both local and global spatial structures while preserving the same output parametrization as the dense controller.

2.1.8 Step Clipping and Gating

At each refinement step, the controller receives the concatenation of the current estimate I_t and the mask M, and predicts (i) a per-pixel correction $dI_t \in [-1, 1]^{3\times H\times W}$ and (ii) a per-pixel gate $g_t \in (0, 1)^{1\times H\times W}$. As mentioned, two controller architectures are supported:

dense and unet. In both cases, the correction is bounded by $\tanh(\cdot)$ and the gate by a sigmoid, ensuring stable ranges for iterative use.

The update is implemented as a gated, damped correction with an explicit trust region enforced by clipping:

$$I_{t+1} \;=\; I_t \;+\; \beta\,\Delta_t, \qquad \Delta_t \;=\; \mathrm{clip}\big(g_t \odot dI_t,\; \pm c_t\big),$$

where $c_t = \texttt{corr_clip} \cdot \texttt{clip_decay}$ is the step-dependent correction budget. The gate functions as a learned "where-to-update" map, suppressing changes in already-consistent regions and focusing updates on uncertain pixels (typically the missing region and its boundary). Clipping ensures that no step can introduce arbitrarily large perturbations, even if proposed by the controller, which is essential for stability over long rollouts.

During optimization, gradient clipping (`clip_grad_norm_` with max norm 1.0) is applied to reduce instability from rare gradient spikes induced by multi-step training. Optionally, a guarded variant of the step (`nftm_step_guarded`) performs backtracking by shrinking β when the inpainting energy increases, improving robustness at the cost of additional forward passes; this mechanism is most useful during inference, where aggressive step sizes may otherwise cause divergence.

2.1.9 Metrics and Evaluation Protocol

Evaluation measures both final reconstruction quality and iterative convergence behavior. The function `eval_steps` runs a rollout of K_{eval} steps and computes step-wise curves for PSNR, SSIM, and LPIPS after each update. To reduce variance from stochastic corruption, the evaluation corruption process is made deterministic by fixing random seeds, ensuring that changes across epochs reflect model updates rather than different masks or noise realizations.

Aggregate evaluation is performed by `evaluate_metrics_full`, which reports both "all-pixels" and missing-region-only scores. Specifically, `psnr_all`, `ssim_all`, and `lpips_all` are computed over the full image, while `psnr_miss`, `ssim_miss`, and `lpips_miss` are computed over the unknown region $(1 - M)$. This separation is necessary because all-pixels metrics can be dominated by the observed region (which is clamped and therefore trivially correct), whereas missing-only metrics isolate the reconstruction difficulty. Finally, distributional realism is assessed using FID and KID implemented via TorchMetrics: features are accumulated over the full evaluation set after mapping images from $[-1, 1]$ to $[0, 1]$, and KID uses a benchmark-dependent subset size to remain reliable on small datasets. In addition to scalar scores, step-wise curves and a `metrics.json` summary are saved to support reproducibility and cross-method comparison.

2.1.10 Flags Reference

For reproducibility we make all important hyperparameters of the inpainting pipeline available as command–line flags in `image_inpainting.py`. Table A.1 in the appendix summarises the most relevant options, grouped by category.[1]

[1] Defaults refer to our final version of the code, with the changes described in Chapter 5. Future work will change these to ones that work best in general.

2.2 Metrics

2.2.1 PSNR

Peak Signal-to-Noise Ratio (PSNR) is a distortion-based measure derived from the mean squared error between a reconstructed image and its ground-truth counterpart [4]. The MSE is defined as:

$$\mathrm{MSE}(x, \hat{x}) = \frac{1}{N} \sum_{i=1}^{N} (x_i - \hat{x}_i)^2 \tag{2.1}$$

where x denotes the ground-truth image, $\hat{x}$ the reconstructed image, N the total number of pixels, and x_i, $\hat{x}_i$ the pixel intensities at location i. MSE measures the average squared pixel-wise reconstruction error.

PSNR is then computed from the MSE as:

$$\mathrm{PSNR}(x, \hat{x}) = 10 \log_{10} \left(\frac{L^2}{\mathrm{MSE}(x, \hat{x})} \right) \tag{2.2}$$

where L is the maximum possible pixel value (e.g., $L = 255$ for 8-bit images or $L = 1$ for normalized images).

It expresses reconstruction fidelity in decibels, effectively rewarding pixel-wise similarity. The metric is simple, fast, and widely used, but it is brutally insensitive to structural or perceptual differences, two images can have the same PSNR while looking completely different to a human observer. In restoration tasks such as inpainting or denoising, PSNR acts as a coarse sanity check: it signals whether the model is collapsing or training correctly, but it does not meaningfully reflect perceptual realism or semantic coherence.

2.2.2 SSIM

Structural Similarity Index (SSIM) evaluates how well structural information, i.e., luminance, contrast, and local texture, is preserved in a reconstruction [5]. Instead of comparing raw pixel errors, it models human perceptual sensitivity by computing similarity over local windows. This makes SSIM more aligned with perceptual judgments than PSNR, especially for artifacts like blurring or contrast shifts. However, SSIM still struggles with large semantic gaps, natural stochastic variation, or high-frequency hallucinations introduced by generative models. In inpainting, SSIM describes whether the model is structurally consistent with the ground truth, but not necessarily whether its completions are plausible or natural.

Under the simplifying assumptions $\alpha = \beta = \gamma = 1$, the SSIM index between two image patches x and y is given by:

$$\mathrm{SSIM}(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)} \tag{2.3}$$

where μ_x and μ_y are the mean intensities of x and y, σ_x^2 and σ_y^2 their variances, and σ_{xy} the covariance between them. The constants C_1 and C_2 are defined as $C_1 = (K_1 L)^2$ and $C_2 = (K_2 L)^2$, with L denoting the dynamic range of pixel values and $K_1 \ll 1$, $K_2 \ll 1$ small constants introduced to avoid numerical instability.

SSIM values typically lie in the range $[-1, 1]$, with 1 indicating perfect structural similarity. In practice, SSIM is computed over local windows and averaged across the image to produce a global similarity score.

2.2.3 LPIPS

Learned Perceptual Image Patch Similarity (LPIPS) measures perceptual similarity by comparing deep feature activations from pretrained networks (e.g., VGG, AlexNet). Instead of relying on hand-designed structural metrics, it leverages the fact that deep features trained for image classification encode texture, shape, and semantic structure [6].

Given two images x and y, LPIPS is defined as a weighted sum of normalized feature distances across multiple network layers:

$$\text{LPIPS}(x, y) = \sum_l \frac{1}{H_l W_l} \sum_{h,w} w_l \left\| \hat{f}_x^l(h, w) - \hat{f}_y^l(h, w) \right\|_2^2 \tag{2.4}$$

where f_x^l and f_y^l denote the activation tensors at layer l of a pretrained network (e.g., AlexNet or VGG) for images x and y, respectively, and H_l and W_l are the spatial dimensions of the feature maps. The feature vectors $\hat{f}$ are channel-wise normalized, and w_l are learned scalar weights calibrated using human perceptual judgment data.

Lower LPIPS values imply closer perceptual resemblance. LPIPS is substantially more aligned with human judgments than PSNR or SSIM, and it has become a de facto standard for evaluating generative and inpainting models. However, LPIPS is ultimately constrained by the biases of the pretrained networks it relies on. Models can exploit this by producing images that match the network's internal feature statistics without actually improving perceptual quality, effectively deceiving the metric.

2.2.4 FID

Fréchet Inception Distance (FID) measures the distance between the distributions of deep features extracted from real and generated images using a pretrained Inception-V3 network [7]. Assuming these feature distributions follow multivariate Gaussian distributions, $\mathcal{N}(\mu_r, \Sigma_r)$ for real images and $\mathcal{N}(\mu_g, \Sigma_g)$ for generated images, FID is defined as:

$$\text{FID} = \|\mu_r - \mu_g\|_2^2 + \text{Tr}\Big(\Sigma_r + \Sigma_g - 2\left(\Sigma_r \Sigma_g\right)^{\frac{1}{2}}\Big) \tag{2.5}$$

where μ_r and μ_g denote the mean feature vectors, and Σ_r and Σ_g the corresponding covariance matrices of real and generated image features, respectively.

Lower FID indicates that the generative distribution is statistically closer to the real one. Unlike PSNR/SSIM, FID does not require paired images and is sensitive to mode collapse. For inpainting, FID captures whether model completions lie on the manifold of natural images rather than just replicating ground-truth pixels. However, FID is notoriously unstable for small datasets, sensitive to preprocessing, and biased by the Inception network's training domain. We implemented FID to assess whether the inpainted images follow the statistical distribution of real images and to evaluate the generative capability of the model. Although originally designed for generative models such as diffusion models, FID remains informative for inpainting because it measures how well the completed images resemble the global characteristics of natural images.

2.2.5 KID

Kernel Inception Distance (KID) measures the discrepancy between the distributions of real and generated images using the squared Maximum Mean Discrepancy (MMD) computed in deep feature space [8]. Features are extracted from a pretrained Inception network, and a polynomial kernel is applied to compare distributions.

Given two sets of feature vectors $\{x_i\}_{i=1}^{n}$ from real images and $\{y_j\}_{j=1}^{m}$ from generated images, KID is defined as the unbiased estimator of the squared MMD:

$$\text{KID} = \frac{1}{n(n-1)} \sum_{i \neq i'} k(x_i, x_{i'}) + \frac{1}{m(m-1)} \sum_{j \neq j'} k(y_j, y_{j'}) - \frac{2}{nm} \sum_{i,j} k(x_i, y_j) \tag{2.6}$$

where $k(\cdot, \cdot)$ is a polynomial kernel defined as:

$$k(x, y) = \left(\frac{1}{d} x^\top y + 1 \right)^3 \tag{2.7}$$

with d denoting the dimensionality of the feature vectors. Lower KID values indicate closer alignment between the real and generated feature distributions.

Unlike FID, its estimator is unbiased, making it substantially more reliable for small datasets, a common scenario in benchmarking sets like CBSD68 or Set12. KID captures both diversity and image quality but is sensitive to kernel choices and therefore less standardized across studies. We implemented KID to complement FID, particularly for scenarios with limited samples where FID becomes unstable.

Chapter 3

Relative works

3.1 Existing Approaches

Nowadays, there are several approaches to tackling the problem of image inpainting, ranging from purely variational methods to highly complex generative deep models. This section describes the main approaches used as baselines, highlighting how they work and their conceptual differences. The three baselines were selected to provide a broad overview of the main paradigms in image inpainting, covering the spectrum from purely mathematical methods to the most recent state-of-the-art generative approaches, as summarized in Table 3.1.

Table 3.1: Conceptual overview of the baseline approaches.

Method	**Key Philosophy**
TVL1	Iterative mathematical optimization, no learning involved
UNet	Learns to inpaint in a single forward pass (one-shot)
Diffusion	Learns to denoise, reconstructing the image iteratively at sampling time

3.1.1 TV-L1 (Total Variation – L1)

The TV-L1 method is a classic optimization approach that does not use any deep learning [9]. Inpainting is formulated as a variational problem in which we seek an image that respects the known pixels while remaining smooth in the missing regions, penalizing abrupt variations through Total Variation (TV). Each image is processed independently through iterative optimization, with no learning involved.

3.1.2 Single-Step U-Net

U-Net is one of the most widely used architectures in computer vision [10]. The model consists of an encoder, which compresses the input into an increasingly abstract representation, a bottleneck where the maximum level of abstraction occurs, and a decoder which reconstructs the image at its original resolution. Encoder and decoder are connected via skip connections, which preserve fine spatial information. The model takes the corrupted image and the mask as input and predicts the complete clean image in a single forward pass.

3.1.3 Diffusion Models

Diffusion Models are one of the most studied approaches in recent years for image generation and inpainting [11]. During the forward process, Gaussian noise is progressively added to the image according to a fixed schedule. The model then learns to predict the noise at each timestep, allowing the model to iteratively reconstruct the clean image during the reverse process. Inpainting emerges at sampling time, where the known pixels are enforced at every step of the reverse process through a strategy inspired by RePaint [12].

Chapter 4

Experimental Setup

4.1 Datasets

4.1.1 Cifar

CIFAR-10 served as the initial dataset for both training and evaluation. It is a widely used benchmark in computer vision, consisting of 60,000 color images of size 32×32 pixels, split into 50,000 training samples and 10,000 test samples. The dataset covers 10 object categories (e.g., airplanes, cars, birds, cats, trucks) and is evenly balanced across them [13]. Because all images are low-resolution (32×32), models must operate under strict spatial limitations, making CIFAR-10 a useful testbed for analyzing how well restoration methods handle fine-grained details with limited pixel information. Its small image size makes it ideal for fast experimentation and clear analysis of localized refinement behavior before scaling to more complex datasets.

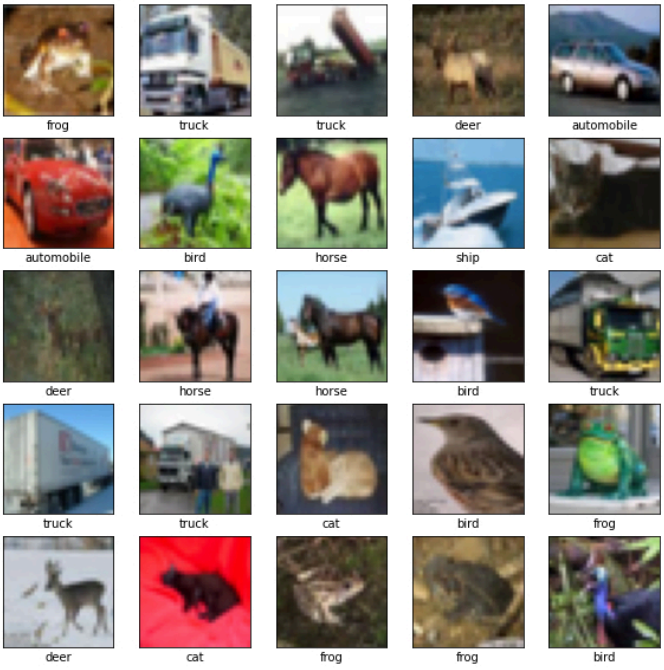


Figure 4.1: Sample images from the CIFAR-10 dataset (32×32 resolution).

4.1.2 CelebAHQ

We added CelebA-HQ to the project to introduce higher-resolution, detail-rich images that assess NFTM's ability to perform localized refinement [14]. We used the resized 256×256

version of CelebA-HQ available on Kaggle, which contains 30,000 high-quality face images derived from the original CelebA dataset but curated for sharper textures, cleaner backgrounds, and more consistent alignment [15]. Unlike CIFAR-10, this dataset provides considerably more fine-grained visual structure, hair strands, facial features, shadows, and skin textures, which enables a far more realistic assessment of how NFTM reallocates computation across complex regions. For experiments that use CelebA-HQ both for training and model evaluation, the dataset is split into an 80% training set and a 20% test set, using a fixed random seed to ensure the split is consistent and reproducible across all experiments.



Figure 4.2: Sample images from the CelebA-HQ dataset (256×256 resolution).

4.1.3 SIDD

To move beyond synthetic noise and evaluate denoising under realistic sensor conditions, we use the Smartphone Image Denoising Dataset (SIDD) [16]. SIDD contains paired noisy and ground-truth images captured from real scenes using several smartphone cameras, covering a wide range of lighting conditions and noise characteristics. Unlike small, low-resolution benchmarks, SIDD images are high-resolution and preserve fine textures and low-contrast details, making it a stringent test of whether iterative refinement can suppress structured noise without oversmoothing. In our setup we use the Medium sRGB subset (`SIDD_Medium_Srgb`), which provides aligned noisy/clean PNG pairs suitable for both quantitative evaluation and qualitative inspection of step-by-step denoising behavior.

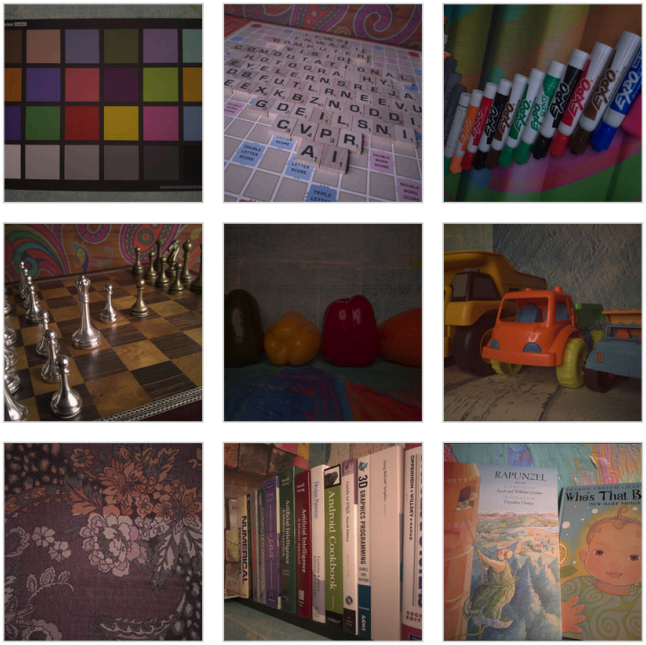


Figure 4.3: Sample images from the SIDD dataset (from 5312 pixels to 2988 pixels resolution).

4.2 Benchmark datasets

4.2.1 CBSD68

We added the CBSD68 dataset exclusively for benchmarking, allowing us to evaluate the model's denoising performance on images it was not trained on. CBSD68 contains 68 color images at the higher resolution of 481x321 (the order of the dimensions simply depends on whether the image is stored in portrait or landscape orientation), featuring natural scenes, objects, and textures with a wide range of spatial frequencies [17]. Because it is used only for testing, CBSD68 provides an unbiased measure of how well NFTM generalizes to unseen visual distributions and noise patterns. This external evaluation helps check whether the adaptive refinement learned from other datasets still works well on new, real-world images.

4.2.2 Set12

We also included the Set12 dataset purely for benchmarking, providing another external test set to evaluate denoising performance independent of the training data. Set12 contains 12 standard grayscale images commonly used in denoising research, featuring simple but diverse structures such as edges, smooth regions, and repetitive textures [17]. Most images in Set12 have a resolution of 256×256, with five higher-resolution samples at 512×512. Since it is used only for evaluation, Set12 helps verify whether the model's adaptive refinement behavior generalizes to grayscale settings and low-texture scenarios, offering a complementary perspective alongside the more complex CBSD68 images.



Figure 4.4: Sample images from the CBSD68 dataset (481x321 resolution).

4.2.3 Noise Types and Corruption Protocol

To ensure fair and reproducible evaluation across datasets (CIFAR-10, CelebA-HQ, CBSD68, Set12), we explicitly define the corruption process used to generate the model inputs from clean ground-truth images.

We distinguish between two fundamentally different noise regimes: masked (non-additive) corruption for inpainting and additive Gaussian noise for denoising.

4.2.3.1 Gaussian Noise

Gaussian noise is modeled as independent pixel-wise perturbations drawn from a normal distribution:

$$\varepsilon_{ijc} \sim \mathcal{N}(0, \sigma^2), \tag{4.1}$$

where σ denotes the standard deviation controlling the noise strength.

A larger σ produces stronger corruption and therefore a more difficult reconstruction problem. The noise is zero-mean, meaning it does not introduce systematic brightness shifts, only random fluctuations.

4.2.3.2 Additive Gaussian Noise

In the denoising setting, Gaussian noise is added to all pixels of the image:

$$I_0 = I_{\text{gt}} + \varepsilon, \tag{4.2}$$

where I_{gt} is the clean ground-truth image and ε is sampled from $\mathcal{N}(0, \sigma^2)$.

This corruption is called *additive* because the noise is directly added to the original image. The entire image is observed but corrupted, and the objective is to remove noise while preserving fine structures and textures.



Figure 4.5: Sample images from the Set12 dataset.

4.2.3.3 Masked Non-Additive Corruption

In the inpainting setting, reconstruction is performed under a binary mask $M \in \{0,1\}^{H\times W}$ indicating observed pixels ($M = 1$) and missing pixels ($M = 0$).

The corrupted initialization is defined as:

$$I_0 = M \odot I_{\text{gt}} + (1 - M) \odot \varepsilon, \tag{4.3}$$

where $\odot$ denotes element-wise multiplication and $\varepsilon \sim \mathcal{N}(0, \sigma^2)$.

In this case, Gaussian noise is injected only into the missing region. The known pixels remain exactly equal to the ground truth.

This corruption is *non-additive* because noise does not modify existing pixels; instead, it replaces the missing region with random values. The reconstruction task therefore becomes conditional completion under a hard observation constraint:

$$I \odot M = I_{\text{gt}} \odot M. \tag{4.4}$$

4.2.3.4 Why the Distinction Matters

Although both settings involve Gaussian noise, they define fundamentally different inverse problems:

- **Denoising (additive noise):** the entire image is corrupted and must be globally restored.
- **Inpainting (non-additive masked noise):** only a subset of pixels is unknown, and reconstruction must remain strictly consistent with observed evidence.

Conflating these two regimes would lead to misleading comparisons. Therefore, in our experiments:

- Additive Gaussian noise is used exclusively for denoising benchmarks.
- Masked non-additive Gaussian corruption is used for inpainting experiments.

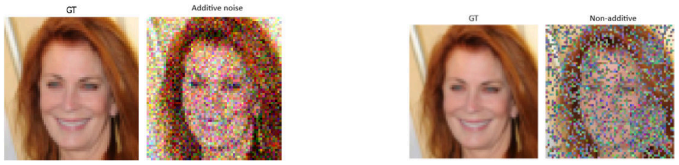


Figure 4.6: Visual difference between additive and non-additive noise. In additive noise the whole image is corrupted while in non-additive, only a fraction of it, including random occlusion blocks.

4.3 Diffusion Baseline Implementation

This section presents the implementation of the baseline based on a DDPM (Denoising Diffusion Probabilistic Models) diffusion model from the paper [11]. The DDPM model and the second U-Net architecture were implemented from scratch, following the tutorial from the course [18].
Its performance on the inpainting task will be analyzed on the two main benchmarks used in the report, CIFAR-10 and CelebAHQ, using the five metrics introduced in the previous section.

4.3.1 DDPM

In this part, the implementation of the DDPM approach used as a baseline diffusion model is described. The process is divided into two main phases: the training phase, in which the model learns to predict the noise through the *forward process*, and the inference phase, in which the *reverse process* is applied using an inpainting strategy inspired by RePaint [12].

4.3.1.1 Training

During the training phase, the *forward process* adds Gaussian noise to a clean image $\mathbf{x}_0$. A key property of DDPMs is the closed-form formulation that allows direct sampling of the corrupted image at timestep t, without explicitly iterating through all intermediate steps:

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \cdot \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \cdot \boldsymbol{\varepsilon}, \quad \boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}) \tag{4.5}$$

where $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$, with $\alpha_t = 1 - \beta_t$. The term β_t follows a linear schedule defined by three hyperparameters: β_{start} (initial noise level), β_{end} (final noise level), and the total number of timesteps T.

The model is trained to predict the noise $\boldsymbol{\varepsilon}$ at each timestep t, minimizing an L1 loss between the predicted noise and the ground-truth noise.

It is important to note that the training is not specialized for the inpainting task, but follows the standard generative diffusion approach. Nevertheless, the model learns to progressively remove noise, which makes it possible to apply it directly to inpainting task. This choice is consistent with the philosophy proposed in RePaint [12], where a generic diffusion model is used without task-specific training.

4.3.1.2 Reverse Process and Inpainting Strategy during Sampling

The *reverse process* starts from pure noise, $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. The model then iteratively applies, for $t = T \to 1$, the following operation:

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \hat{\boldsymbol{\varepsilon}} \right) + \sqrt{\beta_t} \cdot \mathbf{z} \tag{4.6}$$

where $\hat{\boldsymbol{\varepsilon}}$ is the predicted noise, $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, otherwise $\mathbf{z} = \mathbf{0}$, and $\sqrt{\beta_t}$ scales the stochastic term according to the current timestep. The stochastic term prevents the process from collapsing to a flat or overly uniform solution, preserving the generative nature of the model.

To adapt the sampling procedure to the inpainting task, a strategy inspired by RePaint [12] was adopted in a simplified form.

At each step of the *reverse process*, the known pixels are replaced with the values of the original image perturbed at timestep $t-1$, while the missing pixels evolve freely according to the model dynamics:

$$\mathbf{x}_{t-1} = \mathbf{x}_{t-1} \cdot (1 - \mathbf{M}) + \text{perturb}(\mathbf{x}_0, t-1) \cdot \mathbf{M} \tag{4.7}$$

where $\mathbf{M}$ is a binary mask with value 1 for known pixels and 0 for missing pixels, and $\text{perturb}(\mathbf{x}_0, t-1)$ denotes the original image $\mathbf{x}_0$ corrupted at timestep $t-1$ via the forward process, i.e. $\sqrt{\bar{\alpha}_{t-1}} \cdot \mathbf{x}_0 + \sqrt{1-\bar{\alpha}_{t-1}} \cdot \boldsymbol{\varepsilon}$, with $\boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. This operation guarantees consistency of the observed pixels at every denoising step, while the model progressively reconstructs the missing region in a plausible and contextually coherent manner.

4.3.2 Architectures

Two U-Net variants were implemented as backbones for the diffusion model. Both share the standard encoder–bottleneck–decoder structure, but differ in terms of capacity and architectural design choices.

4.3.2.1 TinyUNetDiffusion

The first architecture is adapted from the Single-Step U-Net baseline. The main difference lies in the input channels: since the model receives only the noisy image $\mathbf{x}_t$, the number of input channels is reduced from 4 to 3. The main addition consists of two temporal embedding modules, injected at the bottleneck and at the first level of the decoder. These modules take the normalized timestep t/T as input and allow the model to modulate its behavior according to the current noise level.

4.3.2.2 BigUNetDiffusion

The second architecture is inspired by the course [18]. It introduces three main differences compared to TinyUNetDiffusion. First, the initial convolutional block includes a residual connection, ensuring that the raw signal $\mathbf{x}_t$ is preserved through the first transformations. Second, both the encoder and the decoder are deeper, with two convolutional blocks per level instead of one. Finally, the bottleneck in BigUNetDiffusion is designed to capture the global context of the entire scene, whereas TinyUNetDiffusion operates mainly at a local level.

It is important to note that BigUNetDiffusion was not introduced as a direct comparison with the other baselines, but rather as an observation of how an architecture specifically designed for diffusion models, with a higher parametric capacity that depends on the input resolution, behaves relative to the others. Across all experiments, the number of epochs and the batch size are kept constant, ensuring a controlled comparison.

4.3.3 Results

All experiments were conducted with a fixed batch size of 256. The remaining training configurations vary across benchmarks and are summarized in Table 4.1. The quantitative results are reported in Table 5.8 and Table 5.9 respectively.

Table 4.1: Training configurations for each benchmark.

Benchmark	**Epochs**	T	**Resolution**	β_{start}	β_{end}
CIFAR-10	30	500	32×32	10^{-4}	2×10^{-2}
CelebAHQ	20	1000	128×128	10^{-4}	10^{-2}

As will be shown in Table 5.8 and Table 5.9, both diffusion models achieve lower performance compared to the other baselines across most metrics. This result is consistent with the nature of the approach: unlike the other baselines, the model was not trained specifically for the inpainting task, but learned the general distribution of clean images. Furthermore, diffusion models typically require a significantly larger number of epochs to converge, as the model must learn to predict noise across all 500 or 1000 timesteps rather than a single mapping. With only 30 or 20 epochs , neither model has reached a competitive level of performance.
Nevertheless, BigUNetDiffusion consistently outperforms TinyUNetDiffusion across all metrics, showing that a deeper architecture with global bottleneck context leads to better results even within the same training budget.

4.4 Experiment Configuration

We configured the arguments of image_inpainting.py so that both the training dataset and the benchmark dataset can be passed directly through the command line, allowing any combination of datasets to be used without modifying the code. This setup makes it straightforward to train on one dataset (e.g., CIFAR-10 or CelebA-HQ) while evaluating on another (e.g., CIFAR-10, CelebA-HQ, CBSD68 or Set12). We also introduced

a parameter that explicitly sets the working image size, allowing the same script to run experiments at 32×32, 64×64, or 128×128 resolution. Any dataset with larger native images is automatically resized to the selected resolution, ensuring a consistent input shape across tasks.

4.5 Experiment Environment Setup

For large-scale training and evaluation, the experiments were executed on the Ruche HPC cluster of the Mésocentre Paris-Saclay, which provides access to a wide range of CPU and GPU nodes, including V100 and A100 GPUs suitable for deep learning workloads. In detail, the gpu V100 (normal) partition allows up to 8 GPUs per user, while the gpua100 partition limits each user to 4 A100 GPUs. Ruche offers a controlled, scheduled environment through SLURM, dedicated storage directories for data and experiments, and a stable software ecosystem for running PyTorch-based models [19].

All computationally intensive runs, such as long NFTM rollouts and multi-epoch training, were performed on Ruche to ensure reproducibility and efficient resource use. In contrast, lightweight tests, quick debugging, and non-heavy exploratory notebooks were run on Google Colab, which allowed rapid prototyping before deploying full jobs to the cluster. This setup gave us the flexibility to develop and test code quickly, while still having access to powerful hardware for the heavier training.

4.6 Experiment Monitoring

During experimentation, we heavily relied on the Weights & Biases (W&B) platform to monitor training runs, track hyperparameters, and compare the behavior of different configurations. The –use_wandb argument in our main script enables W&B logging, allowing each experiment to automatically record the hyperparameters uses, losses, PSNR, SSIM, LPIPS, FID and KID metrics, controller settings, and step-wise evaluation curves. Because the script exposes a wide range of hyperparameters, e.g. dataset choice, masking settings, rollout length, controller architecture, noise levels, image size, and other configurations, the W&B interface becomes essential for organizing and inspecting how each change affects performance. Instead of manually parsing log files, we can directly view trends across epochs, visualize metric progressions, and detect unstable runs or poor hyperparameter combinations in one organized place.

This setup also makes systematic experimentation easier. Each run is stored as an entry in the W&B workspace of one particular project, where we can compare multiple projects, filter them by arguments, and identify which configurations yield better refinement or stronger generalization across benchmarks. In addition, we can also monitor compute budgets, iteration time, GPU utilization, and memory consumption. In practice, this allows us to iterate faster, avoid redundant runs, and maintain a reproducible record of all experiments conducted throughout the project.

Chapter 5

Experiments and Evaluation

5.1 Initial Setup

At the beginning of the project we started from a relatively simple NFTM-style iterative inpainting baseline. The core training script `image_inpainting.py` implements a recurrent denoising procedure where a small controller network predicts pixel-wise corrections over K refinement steps. The model is trained end-to-end with an MSE data term on the reconstructed image and optional regularisers (total variation and a contractive penalty).

5.1.1 Baseline configuration

Our initial experiments were conducted on CIFAR-10 at 32×32 resolution. In our initial experiments, we used the following default hyperparameters:

- **Dataset and corruption.** Training and evaluation were performed on CIFAR-10 with random inpainting masks. For each image a missing-pixel ratio p was sampled uniformly from $[0.25, 0.50]$ (`pmin=0.25`, `pmax=0.5`), optionally with additional rectangular occlusion blocks with probability `block_prob=0.5`. Missing pixels were filled with Gaussian noise of standard deviation `noise_std=0.3`.
- **Architecture.** We used the `TinyController` (fully connected "dense" controller) with width `width=48` and 4 input channels (current image, corrupted observation, mask, and residual). At this stage there was no U-Net controller and no multi-scale / pyramid processing; all runs were single-scale at 32×32.
- **Rollout schedule.** During training the rollout length is governed by a simple curriculum: `K_train=8` is the target depth, with the effective depth in epoch e growing from a small base (`K_base=4` inside `train_epoch`) towards `K_train`. Evaluation was done with a fixed `K_eval=12` (default as everything else described here) refinement steps.
- **Step size and clipping.** The iterative updates use a scalar step size β with an epoch-dependent schedule: `beta_start=0.28`, `beta_anneal=0.03`, capped at `beta_max=0.6`. At evaluation we add a small bonus `beta_eval_bonus=0.05` to slightly over-step the training value. Each step is clipped by `corr_clip=0.10` to stabilise the optimisation.
- **Optimisation and regularisation.** We trained with AdamW (`lr=2e-3`, `weight_decay=1e-4`) and batch size `batch_size=128` for `epochs=10`. A small total variation penalty (`tv_weight=0.01`) is applied on the final reconstruction, and

a contractive penalty (`contract_w=1e-3`) encourages small changes between successive iterations. At this stage the descent guard (backtracking line search) was enabled only at evaluation time; `guard_in_train` was left `False` to keep training faster. It turns out that this feature was not useful given that the current setup took very conservative steps, but this will be discussed later sections in detail.

- **Metrics and logging.** After each epoch we ran a full evaluation over the test set and recorded the step-wise PSNR curve ("PSNR vs. refinement step"), together with the final SSIM and LPIPS. These curves and summary statistics were written to `metrics.json` and visualised as PNG plots. At this point, additional details were missing that we needed to fully understand the behaviour over epochs.

To make our ablations reproducible we wrapped the training script in two small drivers, `run_experiments.py` and `run_experiments_sweep.py`. The first script defines a single base configuration (matching the defaults above) and then launches a few simple variants (e.g. changing `K_eval` from 12 to 20, or tweaking `corr_clip` and the β schedule). Each run produces a separate `save_dir` under `runs/`, with its own `metrics.json` and metric curves over the evaluation steps.

5.1.2 Controller architectures

The core NFTM inpainting model is defined in `controller.py`. Both controllers share a common interface

$$(I_t, M) \longmapsto (\Delta I_t, G_t, \log \sigma_t),$$

where I_t is the current iterate, M is the binary mask of known pixels, ΔI_t is a per-pixel RGB correction, $G_t \in (0, 1)$ is a per-pixel gate, and $\log \sigma_t$ is retained for backwards-compatibility with the original heteroscedastic loss.

Tiny dense controller. The original baseline uses `TinyController`, a very small convolutional network:

- Input: concatenation of the current iterate and mask, `concat(I, M)` with shape $(B, 4, H, W)$.
- Body: three 3×3 conv–GELU blocks with a shared channel width `width=48`.
- Heads: three separate 3×3 convolutions producing (i) RGB correction `dI` (tanh-clamped), (ii) a 1-channel gate (sigmoid), and (iii) a 3-channel `log_sigma`.

This controller is deliberately tiny (about the same parameter count as the Tiny U-Net baseline described below) and is used in the NFTM rollout via the `nftm_step` and `nftm_step_guarded` functions in `engine.py`.

UNet-based controller. To give the iterative NFTM model more expressive power without changing the overall optimisation scheme, we also consider a `UNetController`. This wraps a small U-Net backbone (`TinyUNet`):

- The U-Net takes a 4-channel input `(I,M)` and produces 7 output channels.

- These are split into RGB correction, a gate, and `log_sigma`: 3 + 1 + 3 channels respectively.
- As with the dense controller, corrections are passed through `tanh` and the gate through `sigmoid`; the final activation of the U-Net is overridden to `Identity` so the controller can control the output non-linearities.

The `base` argument of `UNetController` / `TinyUNet` controls the channel width of the U-Net (encoder depth and bottleneck). In the main inpainting script we automatically adjust `unet_base` so that the UNet controller has approximately the same number of trainable parameters as the dense controller, so that the comparisons are fair.

5.1.3 Additional baselines: one-shot U-Net and TV-L1

Besides the iterative NFTM controller, we also have two other baselines:

One-shot Tiny U-Net. The script `train_unet.py` trains a standalone Tiny U-Net that performs single-step inpainting:

- Input: `concat(corrupted_image, mask)` with 4 channels.
- Output: an RGB reconstruction in $[-1, 1]$ using a `tanh` activation.
- Architecture: a very small 2-level U-Net with base width `base=10` by default, chosen to have roughly 4.6×10^4 parameters (similar to the NFTM controllers).

The training loss is an ℓ_1 reconstruction term plus a TV regulariser, `tv_weight` $\cdot \mathrm{TV}(I)$. We also track validation PSNR during training. The best checkpoint is saved as `ckpt.pt`.

For evaluation we use `eval_unet.py`, which:

- loads the Tiny U-Net checkpoint,
- applies the same random mask-and-noise corruption as NFTM,
- computes PSNR, SSIM, LPIPS, and FID/KID on the chosen benchmark, and
- writes a `metrics.json` file plus a creates a reconstruction gird as in all other cases (so we can compare).

Classical TV–L1 inpainting. To have a non-learned, optimisation-based reference we also have a TV–L1 baseline in `baselines/inpainting/tvl1_baseline.py`. This uses a primal–dual solver for anisotropic TV-L1:

- The input is the same noisy, masked image I_0 and mask M as in the learning-based models.
- The algorithm iteratively updates an image I and two variables (p_x, p_y) for a fixed number of iterations (`iters`, e.g. 250–400).
- Hyperparameters `lam`, `tvw`, `tau_p`, `tau_d` control the strength of the data term $\lambda \| M \odot (I - I_0) \|_1$ and the TV regularisation, as well as the step sizes.

The TV–L1 script is very similar to the other baselines: it runs on the chosen benchmark dataset, computes global and "missing-region" metrics (PSNR/SSIM/LPIPS on all pixels vs. only the corrupted pixels), and also collects FID/KID statistics. As in other cases, all of the metrics are saved under `runs/`, so we will not repeat them here.

5.1.4 One-click experiment orchestration

The script `run_all_inpainting.py` is an orchestration script that automates a full, reproducible benchmarking pipeline for image inpainting, with a particular emphasis on evaluating the NFTM approach against strong baselines. Its role is not to implement the inpainting method itself, but to coordinate training, evaluation, and comparison across multiple methods under consistent experimental settings. From a single command, it configures datasets, image resolution, masking, device selection, random seeds, and hyperparameters, then sequentially executes all required stages while enforcing a standardized directory structure and logging protocol.

In detail, the script trains and evaluates a standard one-shot UNet inpainting baseline and a classical TV-L1 optimization baseline, ensuring fair comparison. Then, it trains and evaluates the default `image_inpainting.py`, another one with customized arguments, and another one with a unet controller and customized arguments. After all runs are complete, it automatically collects quantitative metrics (PSNR, SSIM, LPIPS, FID, KID, runtime) from each method, aggregates them into JSON and CSV summaries, and prints formatted comparison tables. In conclusion, `run_all_inpainting.py` evaluates the `image_inpainting.py` head-to-head with both deep learning and variational baselines under identical conditions.

5.2 Successful and Failed Additions

At the start of the project, the NFTM inpainting repository we forked was still under active development and fairly disorganised: configuration was scattered across scripts, several code paths were duplicated or unused, and documentation was minimal. During these initial weeks our work was therefore focused mainly on understanding the codebase, reproducing the baseline runs, and mapping how the different components interacted, rather than immediately adding new features. Only once the work on the code finished and we managed to stabilise it and we had a reliable baseline setup were we able to introduce new ideas, fix existing bugs, and clean up the implementation. The remainder of this section summarises these successful and unsuccessful additions.

5.2.1 Coarse-to-fine Inpainting (Pyramid)

One of the main architectural changes we introduced was a multi-scale pyramid rollout for NFTM inpainting. This approach was inspired by other papers we read during our research phases where this method was used successfully [20, 21].

Instead of running all NFTM steps at the native resolution, we run the controller on a coarse-to-fine image pyramid. At each level we perform a small number of NFTM updates at the current spatial resolution, then upsample the prediction, clamp the known pixels to the ground-truth image, and continue at the next finer scale.

The training and evaluation scripts now accept two additional arguments: `--pyramid` (a list of spatial resolutions, e.g. `16,32,64`) and `--pyr_steps` (the number of NFTM steps to allocate to each level, whose sum matches `K_eval`). For this to work, we needed to add helpers for downsampling and upsampling both images and masks, as well as a min-pooling downsample for the binary mask so that any unknown pixel at high resolution remains unknown at coarser scales.

The rollout schedule is made scale-aware. If S denotes the current resolution and $S_{\max}$ the finest resolution, we scale the step size and clipping as

$$\beta_S = \min\big(\beta \cdot S_{\max}/S,\, \beta_{\max}\big), \qquad \texttt{corr_clip}_S = \texttt{corr_clip} \cdot S_{\max}/S.$$

This basically means larger and less tightly clipped updates at coarse scales, where everything is smoother, while still keeping conservative steps at the finest resolution.

Initially, the pyramid behaved as intended: at low resolution the model quickly recovers global structure inside large missing regions, and the high-resolution levels are then used mainly to refine texture and edges.

The success of this approach in our initial experiments turns out to be mainly due to the fact that we had an issue with the base parameters, where we were ultimately taking too conservative steps, so by using this approach, we emulated the a more aggressive behaviour because of the courser levels. We will explain this in detail in Section **??**.

The reality is that we still have problems with the approach, mainly related to the PSNR dropping (along with all other metrics) in the upsampling phases. This of course is more subtle when we are taking less aggressive steps, so we can conclude here that we should use more conservative values for the betas, clipping and regularisation than in the base cases.

One example of the downsides of too aggressive steps and the problems that can occur during the upsampling phases is visible on Figure 5.1. Note that this is a particularly bad example that is intended to showcase the potential problems of this approach if we are not careful with the tuning the other hyperparameters to this approach.



Figure 5.1: CelebA-HQ Dataset Peformace on the First 24 Steps with a `-pyramid 16,32,64` over `-pyr_steps 4,8,18` (64×64 resolution).

Despite the current issues, this approach can be extremely useful for higher image sizes (e.g. 256 × 256, 512 × 512 and even above), where running long single-scale rollouts can be less stable, more expensive computationally or much slower to converge.

5.2.2 Step-Loss

In the baseline NFTM code the data loss is computed only on the final iterate $I^{(K)}$:

$$\mathcal{L}_{\text{data}} = \|I^{(K)} - I_{\text{gt}}\|_2^2.$$

Intermediate steps are not supervised directly.

We tried a step-wise loss that also penalises intermediate iterates. For a rollout of length K_{curr} we assign linearly increasing weights $\alpha_k \propto k$ and collect

$$\mathcal{L}_{\text{step}} = \sum_{k=1}^{K_{\text{curr}}} \alpha_k \|I^{(k)} - I_{\text{gt}}\|_2^2.$$

This can be used by one of the flags `-step_loss` with two options: `final` (original behaviour) and `linear` (step-wise loss). The inspiration came from the fact that gradient problems (vanishing/exploding) can arise given our current architectures [22].

In practice, due to a small bug in the training loop the final-step MSE is still applied on top of the accumulated loss, so our main experiments effectively use the original `final` setting, which led to results only varying due to the lack of seeds for the noise we added. After December, when we fixed the main issue, this type of loss had a similar behaviour in performance improvement to batch normalization, with the initial steps showing better values in quality metrics, but the end results was the same, or even worse. We anticipated that it would help in regularization, but it was not the case.

5.2.3 Residual Connections

5.2.3.1 Overview

This part analyzes the introduction of residual connections within the dense model. Residual connections are widely used in modern deep learning models because they facilitate optimization and improve gradient propagation, reducing the risk of vanishing gradients, particularly in recursive or deep architectures. In the proposed framework, the notion of depth is mainly related to the temporal dimension of the model, since the controller is applied repeatedly over multiple iterative rollout steps, rather than to the architectural depth in terms of stacked layers.

The principle behind this technique is to allow a block in the network to learn an additive correction to its input, rather than having to completely reconstruct the output. In a convolutional network, this translates into a direct sum between the block input and the convolutional transformation output, an operation that preserves pre-existing information and stabilizes the learning process.

It is important to note that our model already features a residual mechanism at the image level. In fact, during the iterative reconstruction process, each new image estimate is obtained from the previous state by applying an incremental correction ΔI. This update is modulated by parameters such as β and the gate, which control the intensity of the update at each iteration. This scheme defines a structured iterative dynamic that allows the model to progressively refine the reconstruction.

In light of this observation, it is therefore interesting to evaluate whether the introduction of standard residual connections within the controller, operating in the feature space,

Table 5.1: Main parameters

Parameter	Value
Epochs	15
K_{train}	24
K_{eval}	24
β_{start}	0.45
β_{max}	0.90

can provide additional benefits compared to the residual dynamics already present in the image.

5.2.3.2 Implementation

To introduce residual connections into the model, the controller was modified by inserting a residual block into the feature space. Specifically, the residual connection was applied after the first convolutional layer.
This choice is motivated by the fact that the initial input of the controller consists of 4 channels (three RGB image channels and one mask channel), while after the first layer, the features are projected into a 48-channel space. Since a residual connection requires the input and output to have the same dimensionality, the sum cannot be applied directly to the raw input. Applying the residual connection in the internal feature space ensures a correct and consistent sum.
The residual block, therefore, implements a transformation of the type:

$$h' = h + F(h) \tag{5.1}$$

Where h represents the output of the first layer and $F(h)$ the subsequent convolutional transformation. The resulting architecture of the controller is illustrated in Figure 5.2.

5.2.3.3 Experiment

The tests were conducted using the parameters shown in Table5.1 and the CIFAR benchmark. The experiment was performed by comparing two configurations: the dense model without internal residual connections and the variant with internal residual connections in the controller.
The experimental results do not show a significant improvement in performance by introducing internal residual connections. During the evaluation phase, the metrics calculated along the iterative rollouts (PSNR, SSIM, and LPIPS) are very similar between the two configurations, as shown in Table 5.2, for example PSNR values during the some specific steps. Even the final performance after the last iteration step shows negligible differences, as shown in Table 5.1.
These results indicate that the starting model is already characterized by good stability thanks to the residual dynamics applied at the image level. The addition of standard residual connections within the controller, therefore, does not provide a marked benefit in terms of final quality, but confirms the robustness of the iterative approach adopted.

Table 5.2: PSNR during evaluation steps

Model	Step 1	Step 5	Step 10	Step 15	Step 20	Step 24
Dense	17.36	26.87	28.32	28.33	28.34	**28.34**
Dense + Residual block	17.36	26.85	28.28	28.30	28.31	**28.31**

Table 5.3: Final evaluation metrics

Model	PSNR (dB)	SSIM	LPIPS
Dense (no residual)	**28.344**	**0.9096**	0.00684
Dense + residual block	28.313	0.9087	**0.00679**

5.2.4 Batch Normalisation

5.2.4.1 Overview

This section analyzes the introduction of Batch Normalization (BN) within the dense controller. Batch Normalization is a technique widely used in deep learning models and, in particular, in convolutional networks, as it allows intermediate activations of layers to be normalized during training.

The main purpose of Batch Normalization is to reduce variations in the distribution of activations between layers, limiting extreme changes in intermediate values. This helps to make the training process more stable, reducing the risk of divergence and facilitating gradient propagation. Furthermore, by normalizing activations, the network can converge more quickly, allowing in many cases to achieve good performance in fewer epochs.

The main effect of Batch Normalization occurs during the training phase, since normalization uses statistics calculated on the current batch (mean and variance). During the evaluation phase, on the other hand, the statistics estimated during training are used, and the role of Batch Normalization is mainly to maintain a distribution of activations consistent with the one learned.

In general, Batch Normalization is also known to have an implicit regularization effect, making the model less sensitive to weight initialization and learning parameter values.

5.2.4.2 Implementation

Batch Normalization was introduced into the dense controller by inserting a normalization layer after each convolutional layer.

Applying normalization after convolution allows the activations produced by the linear layer to be normalized before applying the non-linear activation function. This configuration is widely adopted because it allows the activation function to operate on inputs with a more stable distribution, improving the robustness and convergence speed of training.

In the proposed model, this structure can be seen in Figure 5.3, where each convolutional layer of the controller is followed by a Batch Normalization layer and the activation function.

Table 5.4: Final evaluation metrics with Batch Normalization

Model	PSNR (dB)	SSIM	LPIPS
Dense	**28.344**	**0.9096**	**0.00684**
Dense + Batch normalization	28.122	0.9048	0.00738

Table 5.5: First epoch metrics with Batch Normalization

Model	PSNR (dB)	SSIM	LPIPS
Dense	21.90	0.7779	0.0520
Dense + Batch normalization	**24.40**	**0.8494**	**0.0193**

5.2.4.3 Experimental results

After introducing Batch Normalization into the dense controller, the model was trained using the parameters shown in Table 5.1, with the exactly set up of the residual experiment, and evaluated on the CIFAR benchmark. As expected, the addition of Batch Normalization does not lead to a significant improvement in final performance in terms of PSNR, SSIM, and LPIPS, as shown in Table 5.4.
However, a relevant aspect emerges when analyzing the training dynamics. In particular, comparing the performance in the first epochs with that obtained in the last evaluation steps, we observe that the model with Batch Normalization achieves higher PSNR values already in the initial stages of training. For example, at epoch 1, there is a difference of about 2.5 dB in PSNR compared to the model without normalization, as it is possible to see in Table 5.5.
This behavior indicates that Batch Normalization promotes faster convergence, allowing the model to achieve comparable performance in fewer epochs, while not significantly improving the final quality of the reconstruction.

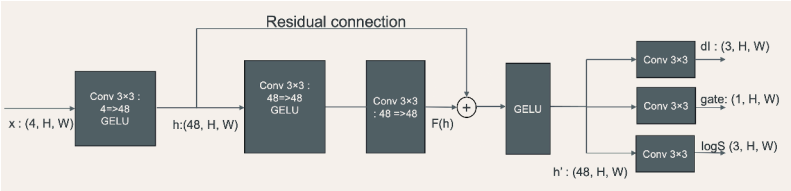


Figure 5.2: Residual connection

5.2.5 RNN Controller

We also investigated a recurrent controller variant to explicitly inject sequential dependencies into the NFTM update rule. The motivation was to investigate whether modeling spatial correlations as a sequence could help propagate global context into the per-step correction, beyond what shallow convolutional stacks provide. Concretely, the RNN controller first encodes the current image estimate and mask with a lightweight convolutional

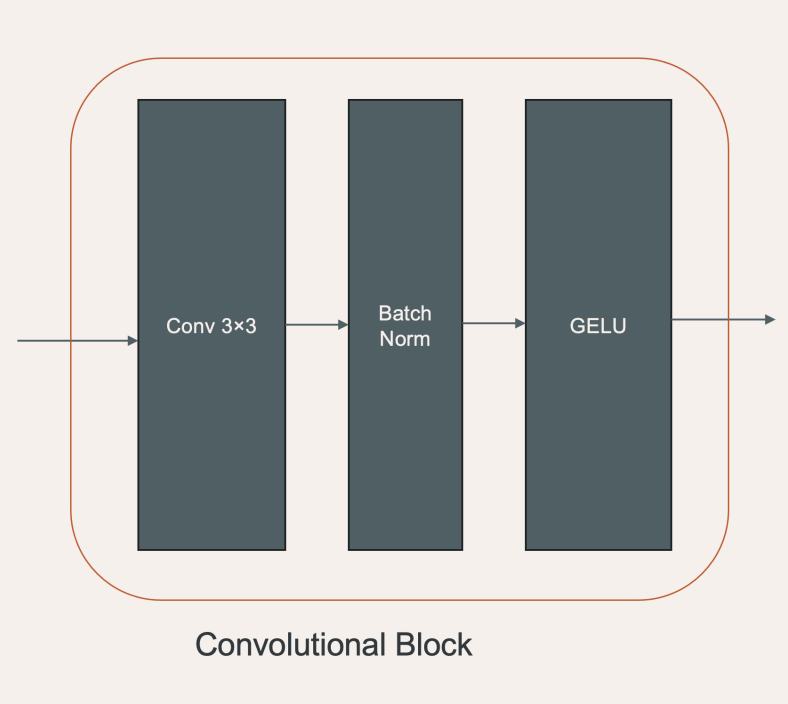


Figure 5.3: Batch normalization

encoder, then flattens the spatial feature map into a sequence and processes it with a GRU. The recurrent output is reshaped back to image space and mapped to the same controller outputs as the other variants: a per-pixel correction field, a gating mask, and a log-variance term kept for interface compatibility. This design was intentionally minimal, aiming to add temporal-style memory over spatial positions without moving to a full transformer or significantly increasing parameter count.

In practice, this direction was deprioritized and ultimately abandoned as the project focus shifted. Early experiments with the initial dense and UNet-based controllers already achieved strong and stable performance, making them a more promising foundation for further work. Rather than expanding the controller design space, we therefore concentrated on systematically improving these baselines and identifying well-optimized parameter settings, focusing effort on tuning rollout depth, step sizes, and gating behavior to fully exploit their capabilities.

5.2.6 SIDD Real Camera Noise Test

5.2.6.1 Motivation and Design Choice

Unlike synthetic denoising benchmarks, the Smartphone Image Denoising Dataset (SIDD) provides real camera noise pairs (NOISY / GT) without explicit corruption parameters. Therefore, synthetic noising strategies used in previous experiments were not applicable.

To handle this setting, a dedicated script `sidd_denoising.py` was implemented, adapting the NFTM iterative framework to real paired noisy–clean data.

5.2.6.2 Scene-Level Split to Avoid Data Leakage

SIDD contains multiple captures of the same scene under different camera and exposure configurations. A naive random split would leak scene content between train and test sets.

To prevent this, scenes were grouped using the *scene number* identifier extracted from folder names. The dataset was then split at the **scene-group level**, ensuring that:

- No scene content appears in both training and testing.
- Different camera instances of the same scene remain within the same split.

This guarantees a content-level generalization test rather than memorization of scene structures.

5.2.6.3 Tile-Based Training Strategy

SIDD images are high-resolution (several megapixels), making full-image iterative rollout computationally prohibitive.

To address this, a tile-based approach was adopted:

- Images are divided into patches of size 64×64.
- A stride of 64 was used during training.
- Tiles are extracted from paired noisy/GT images.

Each tile is normalized to $[-1, 1]$ and processed independently during training.

Since denoising differs from inpainting, the mask is set to zero everywhere:

$$M = 0,$$

meaning no hard-known region constraint is enforced. The rollout operates over the entire image.

Additionally, an optional per-epoch subset sampler was introduced to randomly resample tiles each epoch, improving diversity while keeping memory and training time bounded.

5.2.6.4 Iterative NFTM Denoising Formulation

For each tile, the initialization is:

$$I^{(0)} = I_{\text{noisy}},$$

and the rollout proceeds for K_{train} steps during training and K_{eval} during evaluation.

The update follows the standard NFTM step:

$$I^{(k+1)} = I^{(k)} + \Delta_\theta(I^{(k)}, I_{\text{gt}})$$

Key parameters used: K_{train}, K_{eval}, β scheduling with optional annealing, correction clipping (`corr_clip`), contractive regularization (`contract_w`), optional descent guard. Loss was computed either only on the final step (default) or linearly weighted across rollout steps.

5.2.6.5 Full-Resolution Reconstruction

For visualization and reporting, full-resolution SIDD images were reconstructed from denoised tiles.

The process:

1. Extract overlapping tiles using configurable patch and stride.
2. Optionally apply reflected padding (tile_pad) to provide context.
3. Run NFTM denoising independently per tile.
4. Remove padding.
5. Stitch tiles together using a smooth separable blending window.

Blending weights are accumulated and normalized:

$$I_{\text{full}} = \frac{\sum_t w_t \odot I_t}{\sum_t w_t}$$

This avoids blocking artifacts and ensures seamless reconstruction.

5.2.6.6 Observed Behaviour (Step-wise Dynamics)

Step-wise evaluation curves show:

- PSNR increases monotonically with rollout steps (Fig. A.13).
- LPIPS decreases steadily as refinement progresses (Fig. A.14).
- Both curves exhibit smooth convergence without noticeable oscillations, indicating stable iterative dynamics.

This confirms that NFTM remains stable even when trained on real camera noise rather than synthetic corruption.

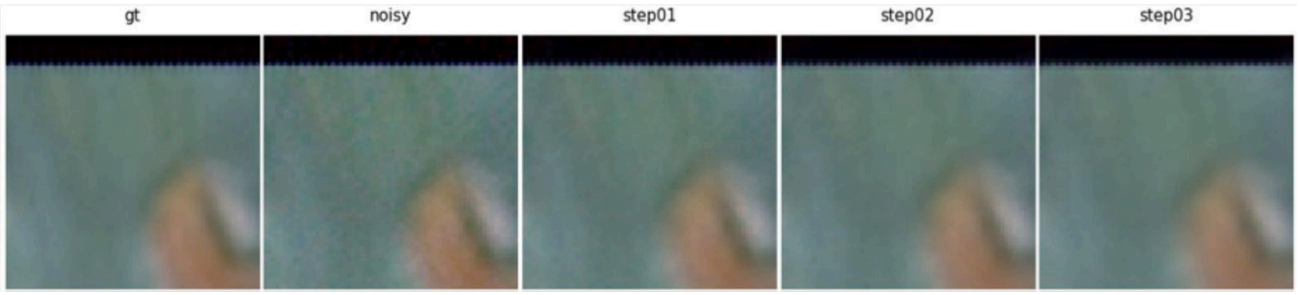


Figure 5.4: SIDD results, at the tile level, resolution 64x64

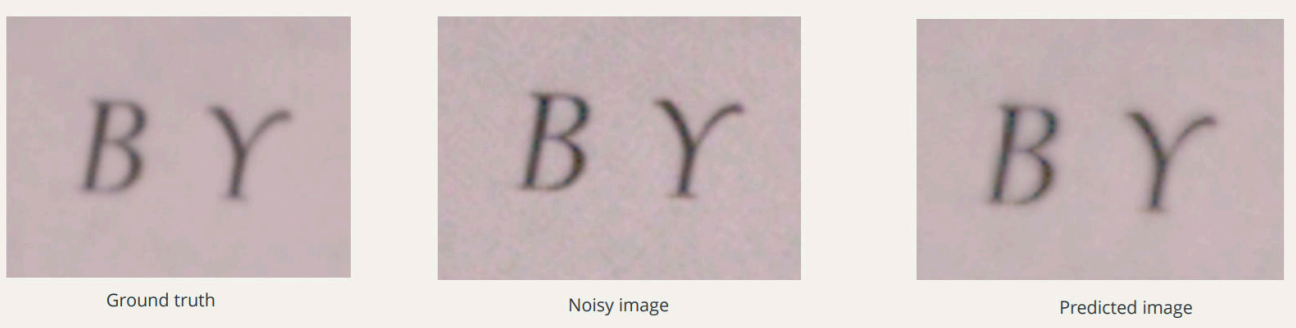


Figure 5.5: SIDD results, after stitching tiles back together - zoomed in text

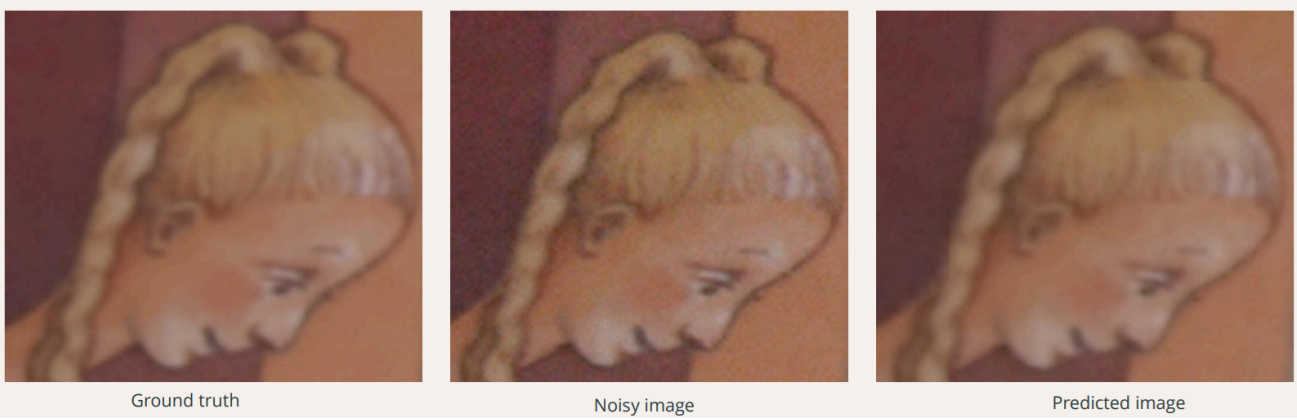


Figure 5.6: SIDD results, after stitching tiles back together - zoomed in drawing

5.2.6.7 Successful Outcome

The SIDD integration can be considered a **successful addition** for several reasons:

- No synthetic noise model was required.
- The iterative controller generalized to real sensor noise.
- Tile-based rollout made high-resolution denoising feasible.
- Full-image reconstructions show visually coherent results.

The best-performing configuration is reported in Table A.8 (Appendix), corresponding to the guarded rollout with linear step-weighted loss and overlapping tile-based reconstruction. Under this setting, the model achieved an evaluation PSNR of **40.86 dB**, final SSIM of **0.9656**, and final LPIPS of **0.0203**. Qualitative results further support this outcome: tile-level denoising at 64×64 (Fig. 5.4) shows progressive suppression of real sensor noise, and the stitched subparts of the full-resolution reconstructions remain visually coherent without obvious seam artifacts, and the characteristic sand-grain sensor noise is largely removed (Fig. 5.5 and Fig. 5.6). Overall, these results confirm that the NFTM iterative denoising formulation remains stable and effective under real camera noise conditions, without relying on any explicit noise model.

5.2.7 Other additions

Besides the main changes, we made several smaller but important improvements to the codebase:

- **Documentation.** We added short README-style notes in a `docs/` folder for some of the main scripts so that everything that we add throughout the project to the codebase is explained properly.
- **Bug fixing and cleanup.** A number of minor bugs, dead code paths and inconsistent defaults (like upsampling the GIFs and the pictures in the `runs/` folder) were fixed, and the repository was cleaned so that only the work related to Image Inpainting stays in the repository and every other component related to the NFTM paper was removed. We also included new and clean `requirements.txt` and `requirements-cuda.txt` files with version conflicts in mind, so that users have the option to set everything up in their local enviroment.
- **Repository README.** We also wrote a new top-level `README.md` that documents the current repository layout, the main inpainting scripts, and how to run single experiments or larger sweeps. It summarises the required datasets, command-line flags (with short explanations for each of them), and expected outputs (`metrics.json`, curves, and visualisations).
- **Deterministic seeds.** After finding fluctuations between the evaluation scores between epochs, we noticed that there was no fixed seed for the noise added for each run. We added it to make the evaluation scores more meaningful.
- **Modular scripts.** As the original monolithic inpainting script was getting too large to navigate, we split it into smaller modules (`engine.py`, `controller.py`, `rollout.py`, etc., inside the `nftm_inpaint/` folder), which makes it easier to test and extend individual components.
- **Higher resolutions.** The full pipeline was extended from CIFAR-10 (32×32) to also support 64×64 and 128×128 inputs, both in the NFTM inpainting script and in the Tiny U-Net and TV–L1 baselines.
- **Epoch logs.** Training now writes one line per epoch to a `train_log.txt` file (model parameters, loss, PSNR, controller settings, number of backtracking steps if guard is enabled, etc.), which we later use to analyse the behaviour between epochs.
- **Configurable visualisation scale.** The visualisation upsampling factor for PNG/GIF outputs is no longer hard-coded; it can be set via `-viz_scale`, which is useful when moving from 32×32 to larger images.

5.3 Experiments not Relevant for the Best Configuration

5.3.1 K_train and K_eval

In this section we analyze the impact of the rollout length used during training (K_{train}) and evaluation (K_{eval}) on the reconstruction performance of the proposed model. The

Hyperparameter	Value
β_{start}	0.28
β_{anneal}	0.03
TV weight	0.01
Correction clip	0.1

Table 5.6: Hyperparameter configuration shared across all experiments.

experiment is conducted using the same evaluation metrics, namely PSNR, SSIM, and LPIPS, and share the same hyperparameter configuration reported in Table 5.6, and on the CIFAR benchmark.

5.3.1.1 Definitions

K_{train} denotes the number of unrolled refinement steps used during training, while K_{eval} denotes the number of refinement steps applied at inference time.

5.3.1.2 Experiment

This experiment is performed using the following combinations, with 15 epochs:

$$K_{\text{train}} \in \{2, 6, 10, 14, 18, 22\}, \quad K_{\text{eval}} \in \{2, 6, 10, 14, 18, 22\}.$$

The corresponding heatmaps are shown in Figure 5.7. As can be observed, the dominant effect on all three metrics is driven by K_{eval}, while variations in K_{train} produce only marginal changes. This indicates that increasing the number of evaluation steps K_{eval} yields greater performance improvements, with minimal sensitivity to K_{train}.

5.3.1.3 Discussion

These results suggest that using differents K_train, they have no impact on the metrics, but only the K_eval impacted them. However, as discussed in the following sections, an alternative explanation is that the hyperparameter configuration reported in Table 5.6 may not be fully optimal, and could limit the observable benefits of larger training rollouts.

5.3.2 Robustness to Different Noise Distributions in Inpainting

To evaluate the robustness of the iterative controller, we performed additional experiments where different noise distributions were applied only at evaluation time, without any retraining of the model.

5.3.2.1 Experimental Protocol

The model was trained using masked Gaussian corruption. During evaluation, the missing region $(1 - M)$ was instead filled using alternative noise distributions, while keeping the mask structure, rollout length, and hyperparameters fixed. No fine-tuning was performed.

The following noise types were tested:

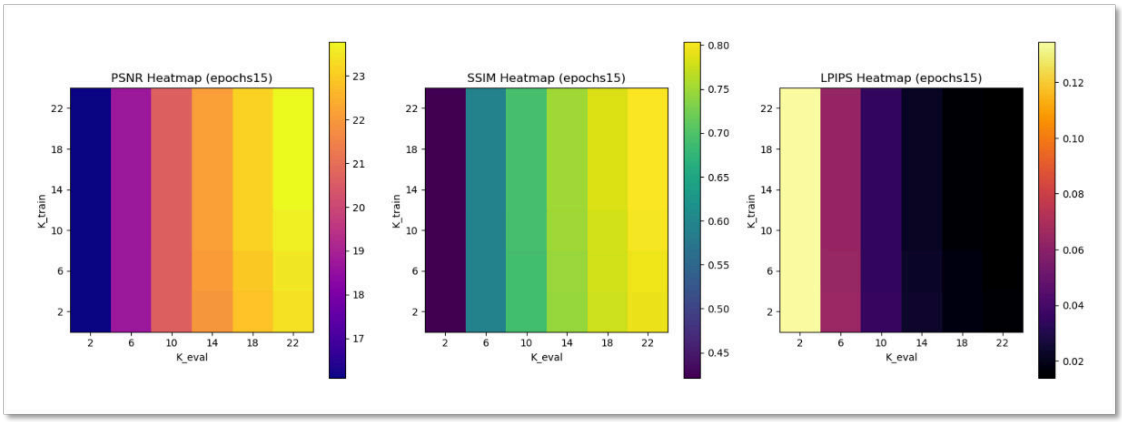


Figure 5.7: Heatmaps of PSNR, SSIM, and LPIPS for $K_{\text{train}}, K_{\text{eval}} \in \{2, 6, 10, 14, 18, 22\}$ trained for 15 epochs.

- Gaussian noise ($\sigma = 0.3$)
- Uniform noise
- Salt-and-Pepper noise (probabilities 0.1 and 0.2)
- Poisson noise (peak values 30 and 10)
- Speckle noise ($\sigma = 0.2$)

In all cases, corruption was restricted to the missing region only, preserving the hard constraint on observed pixels.

5.3.2.2 Quantitative Results

Table 5.7 reports the evaluation metrics across noise types.

Noise Type	PSNR↑	SSIM↑	LPIPS↓	FID↓	KID↓
Gaussian ($\sigma = 0.3$)	30.0207	0.9380	0.01547	10.840	0.01001
Uniform	30.0192	0.9380	0.01547	10.837	0.01001
Salt&Pepper (0.1)	30.0302	0.9380	0.01547	10.837	0.01001
Salt&Pepper (0.2)	30.0223	0.9380	0.01548	10.840	0.01001
Poisson (peak 30)	30.0249	0.9378	0.01549	10.930	0.01004
Poisson (peak 10)	30.0251	0.9378	0.01549	10.930	0.01004
Speckle ($\sigma = 0.2$)	30.0468	0.9381	0.01545	10.839	0.01001

Table 5.7: Evaluation metrics under different noise distributions applied only to the missing region.

Across all corruption types, the metrics remain nearly identical. The maximum observed variation is:

- PSNR variation: less than 0.03 dB,

- SSIM variation: less than 3×10^{-4},
- LPIPS variation: less than 5×10^{-5},
- FID variation: approximately 0.09,
- KID variation: approximately 3×10^{-5}.

These differences are negligible and fall within normal stochastic variation.

5.3.2.3 Interpretation

The near-identical results across Gaussian, Uniform, Salt-and-Pepper, Poisson, and Speckle noise confirm that the controller is largely insensitive to the statistical distribution of the corrupted region.

Despite being trained exclusively with Gaussian noise, the model converges to essentially the same reconstruction regardless of the initialization distribution inside the mask. This indicates that:

- The controller does not rely on modeling the corrupted pixels themselves.
- The iterative dynamics are driven primarily by contextual information from the observed region.
- The rollout converges toward a context-consistent fixed point that is largely independent of the initial noise realization.

An important implication of this behavior is that the method behaves primarily as an inpainting approach, rather than exploiting potentially informative corrupted pixels. For example, multiplicative speckle noise retains weak signal structure within the corrupted region, and Poisson noise is signal-dependent; however, the model does not extract measurable benefit from these residual structures. Instead, it effectively treats the missing region as uninformative and reconstructs it solely from contextual cues.

This observation reinforces the interpretation of the NFTM controller as implementing a robust, context-driven refinement process rather than a noise-distribution-specific denoiser. The corrupted region mainly determines the starting point of the rollout, while the final reconstruction is governed almost entirely by surrounding visual evidence and the learned update dynamics.

5.4 Analysis of the Most Influential Components

As a reminder, we detail our initial NFTM architecture over one step in Figure 5.8. In this section we analyse the most influential components and how they interact with each other and how they effect the convergence, training speed, rollouts, and the final results.

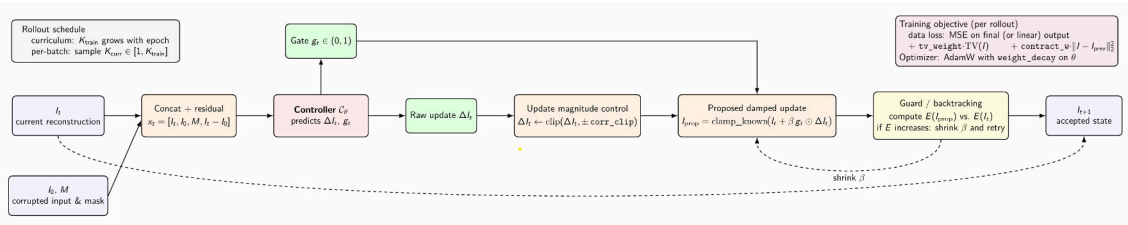


Figure 5.8: Final inpainting step and training pipeline (fixed rollout; no gate/β; no guard/backtracking; optional `corr_clip` as a pure per-step limiter).

5.4.1 β and Rollout Types

The initial NFTM update

$$I_{t+1} = I_t + \beta \, \text{gate}_t \odot dI_t$$

is controlled by the step size β and by several regularization terms (TV weight, correction clipping, contractive loss). With the default hyperparameters from the original code (still the same in our finalized code for reference), we observed that rollouts converged very slowly: even when we increased K_{eval} well beyond the training horizon.

At first, we were told that the NFTM controller might keep improving "beyond" the training horizon, i.e. that more steps would always help. Our experiments actually showed the opposite: with the default step sizes and regularization, the system actually takes too conservative steps, so the conclusion is not that it keeps improving, but rather that the steps were so small that it was not able to finish the inpainting process (this goes for training as well). The limitation was therefore in the hyperparameters, not in the architecture itself. A picture of this behaviour on the CelebA-HQ dataset is visible in Figure 5.9.



Figure 5.9: CelebA-HQ Dataset Peformace with Default Parameters (64×64 resolution).

To address this, we ran several experiments with hyperparameters that led to more aggressive steps and less regularisation on both the CIFAR-10 and CelebA-HQ datasets, using both the DENSE and U-NET controllers. After we got an intuition for what ranges work the best, we set up a dedicated hyperparameter search over step sizes, rollout depth,

and regularization strengths. The sweep is implemented as a Bayesian hyperparameter optimization using Weights & Biases, targeting final evaluation PSNR as the objective. The resulting configuration led to much faster convergence and a large improvement in final image quality across all metrics.

However, it turns out that even with this advanced setup (relying on the progress and information we had until December), we were still far from the best results and performance our architecture is able to achieve. In our experiments, the gate mechanism, the β-based damping schedule, and the different rollout strategies (e.g., curriculum vs. fixed rollout depth) did not provide a consistent improvement in final reconstruction quality once the update magnitude was properly controlled through the other parameters. Their practical effect was mostly regularization: they reduced the effective step size of each iterative update and made the optimization dynamics more conservative, but they did not improve the final optimum reached by the model at all.

More specifically, both the gate and β are basically multiplicative scaling factors on the predicted correction. As a result, their main contribution in our setting was to slow down or dampen the update process rather than to improve the direction of the updates. Similarly, rollout variants mainly changed the training dynamics (stability and convergence speed) but did not lead to better final PSNR/SSIM/LPIPS when the remaining core components were well tuned.

Our final conclusion is that these components are not fundamentally useless, but in our setup they are quite redundant with stronger and more direct controls of update magnitude (we discuss these later in this section). Therefore, we treat them primarily as forms of stabilizers rather than mechanisms that drive performance.

5.4.2 `corr_clip`

One of our main findings is that the dominant factor controlling the update magnitude is not the gate or β, but `corr_clip`. This became clear after looking at the update rule: the controller predicts a correction ΔI, and `corr_clip` clamps this correction element-wise before it is applied. In practice, this leads to a hard bound on how much a pixel can change in one iteration, which makes it the first (in the order of controls applied) and strongest control over step size.

On the other hand, the gate and β only scale the update after the controller prediction is produced. They can reduce the update magnitude, but they do not constrain extreme predictions as directly as `corr_clip`. As a result, `corr_clip` had a much larger effect on rollout behavior, convergence speed, and the shape of the PSNR-vs-step curves.

We also modified the implementation so that setting `corr_clip=0` disables clipping (instead of clamping all corrections to zero, which would freeze the iterative process and produce no updates). This change makes the flag more intuitive and allows controlled ablations between "clipped'" and "unclipped" dynamics.

Interestingly, after tuning the remaining components (and especially in our best inpainting runs), removing clipping did not hurt final performance and in some cases even simplified optimization. For this reason, we do not treat `corr_clip` as necessary; rather, we view it as a strong stabilization/control mechanism that is influential, but not always beneficial in the final best-performing configuration.

5.4.3 `K_train` and `K_eval`

Another important conclusion concerns the rollout length parameters `K_train` and `K_eval`. In the original setup, training and evaluation do not necessarily match: training may use a curriculum (progressively increasing rollout depth), random rollout lengths, and optional β schedules, while evaluation is usually performed with a fixed number of steps (`K_eval`). This obviously creates a train–test dynamics mismatch.

Curriculum and rollout sampling. By default, training uses a curriculum on the rollout depth:

$$K_{\text{train}}^{(\text{epoch})} = \min(K_{\text{target}}, K_{\text{base}} + \text{epoch}),$$

with a small initial rollout length. In our implementation, this means early epochs start with short rollouts (e.g., around 5 steps in epoch 1 when `K_target=10`), and only later epochs expose the model to longer trajectories. In addition (disabled by default), rollout depth can be sampled per batch (random rollout length), which changes the training dynamics across iterations even more.

This design is reasonable in principle (it should make optimization easier at the beginning), but in our experiments it just led to weaker control of later rollout steps. In practice, the model learned to perform most of the useful correction in the first few steps, which meant that later steps contributed much less.

Why the PSNR curves flatten after 5–6 steps in our earlier experiments. This explains the shape of our PSNR-vs-step curves: a large gain in the first 5–6 steps, followed by a strong slowdown (or near-plateau). The model is initially trained mostly on short trajectories because of the curriculum, so it learned strong early corrections first. Steps beyond the initial rollout length are only trained later and less consistently, and hence they are less optimized. As a result, the early part of the trajectory becomes the "main working regime", and later steps behave more like small refinements.

Role of variable β. We also experimented with β schedules (start / max / annealing / eval bonus), which were used to modulate the effective step size across epochs and between training and evaluation. However, in our final observations, these schedules mostly acted as regularization/damping rather than providing a clear accuracy benefit (as explained earlier). Moreover, if `K_train` is already changing through curriculum and random rollout sampling, adding a changing β introduces an additional source of train–test mismatch.

Why fixed rollout works best. The most stable and best-performing configuration in our final runs was to make training and evaluation dynamics match as closely as possible:

- fixed rollout length during training (`-fixed_K`),
- no rollout curriculum (`-no_curriculum`),
- and `K_train = K_eval`.

This makes sense intuitively as well: the model is optimized for exactly the same number of iterative updates that it will use at test time. In other words, every step in the rollout is trained from the beginning, instead of relying on later epochs to "catch up" and learn the tail of the trajectory.

Implementation changes from the conclusions of this subsection. To support controlled ablations and remove the curriculum mismatch, we added two flags:

- `-fixed_K`: disables random rollout-length sampling and always uses the current training rollout length,
- `-no_curriculum`: disables the rollout curriculum and uses `K_train = K_target` from epoch 1.

Together, these flags allow us to train directly on the same rollout depth used at evaluation, which was consistently the strongest setting in our final experiments.

5.4.4 Learning Rate α

The learning rate α turned out to be one of the most important hyperparameters for denoising experiments, especially for the dense controller. In this case, we saw an interesting failure mode: training appears normal for several epochs, then the PSNR suddenly drops and the iterative rollout starts to degrade the image instead of improving it.

Our final interpretation is that this is primarily a dynamical stability issue rather than a standard overfitting issue. In particular, when the effective update map becomes too aggressive, small errors are amplified from one iteration to the next. In practice, this can happen when the optimizer pushes the controller parameters into a regime where the local Jacobian gain is too large (see the next subsection). The result is a sudden collapse in multi-step denoising performance.

A lower learning rate is basically a strong regularizer against this behavior:

- it reduces how quickly the optimizer can move the parameters into unstable regions,
- it also reduces the probability of sudden PSNR collapse in denoising,
- and in our runs it often does so with only a small change in convergence speed (or no major practical slowdown).

This was one of the most useful practical fixes for dense-controller denoising: reducing α below the default value can stabilize training without sacrificing anything on the final performance.

Dense vs. UNet controllers. Another important observation is that the optimal learning-rate regime depends on the controller architecture:

- **Dense controller:** more sensitive to learning rate in denoising; larger α can trigger collapse.

- **UNet controller:** in our denoising experiments, it tolerated (and sometimes benefited from) a higher learning rate than the dense controller, even higher than the default `2e-3`, while remaining stable.

This suggests to us that the UNet architecture has a more stable inductive bias for iterative denoising in our setup (e.g., smoother/structured corrections), whereas the dense controller reaches unstable update dynamics more easily when trained aggressively.

Practical takeaway. For denoising tasks, the learning rate should be looked at as a stability parameter, not only as an optimization-speed parameter. Moreover, architecture-specific tuning is necessary: a value that is stable for UNet may cause collapse for the dense controller.

5.4.4.1 Likely Reason for the Collapse: Jacobian Spectral Radius

To investigate the collapse, we added a Jacobian logging procedure that estimates the local gain of the iterative update map on sampled batches during training. The goal is to approximate whether the rollout behaves locally like a contraction or an expansion.

Conceptually, if the update map is denoted by F, then instability is expected when the local Jacobian norm becomes too large (informally, "spectral radius / gain above 1"), meaning that perturbations can grow across iterations instead of shrinking. In that regime, each NFTM step most likely amplifies errors, which explains the observed PSNR collapse in denoising.

Important note. What we log is not the exact global spectral radius of the full training dynamics. Instead, we estimate a sampled local Jacobian gain (with power iteration on selected mini-batches and selected rollout states). Therefore, it should be interpreted as a **proxy for investigation**, not a formal proof of stability/instability.

Despite this limitation, the signal is very useful in practice:

- before collapse, the estimated Jacobian gain typically increases,
- around collapse, it can spikes dramatically,
- and this correlates well with the sudden drop in denoising PSNR.

Implementation support for logging the estimated Jacobian gain. We added flags to make this observable during training:

- `-log_jacobian`: enable per-epoch Jacobian logging,
- `-jacobian_K`: number of rollout steps in the probed map (e.g., 1-step or 3-step map),
- `-jacobian_batches`: number of sampled batches to average,
- `-jacobian_power_iters`: number of power-iteration steps for the estimate.

This allowed us to observe the collapse process (at least through sampled local Jacobian estimates), which supports our conclusion that the denoising failure mode is caused by unstable iterative dynamics rather than other causes.

5.4.5 Guard and Backtracking

Our initial guard mechanism was designed around β-based backtracking (i.e., reducing the step size when an update increased the energy, this was detailed in earlier sections). After our later findings, this design became less meaningful, because β was not the main factor controlling update magnitude in our setting (the dominant factor was `corr_clip`).

For this reason, we changed the implementation from a classical backtracking scheme to a simpler and more robust descent check:

- We propose a single NFTM update.
- We compute the chosen energy (data term + optional TV term) before and after the update.
- If the energy decreases, we accept the step.
- If the energy increases, we reject the step.

In other words, the guard is now a **one-step accept/reject mechanism** instead of a true multi-try backtracking loop. This also makes it less dependent on β tuning.

Training vs. evaluation behavior. We also used the rejection behavior depending on the phase:

- **Evaluation:** strict rejection (the state is kept unchanged if the step increases energy).
- **Training:** straight-through style rejection (forward pass keeps the old state, but gradients still flow through the proposed step), so training does not stall when many steps are rejected.

This change was necessary for stable optimization, but it also highlights an important limitation: the guard only forces a local condition on each individual rollout step.

Final takeaways. In our experiments, the guard/backtracking mechanism provided little to no meaningful improvement in final performance. The reason is structural. The guard helps per step: it can reject updates that increase the chosen energy. However, it does not constrain the optimizer itself. Therefore, it cannot prevent training from moving the controller parameters into an unstable region of parameter space.

As a result, the guard can mask some problems locally, but it does not solve the underlying dynamical instability problem. In that sense, the backtracking mechanism (in its original or modified form) was mostly not useful in our best-performing configurations.

5.4.6 Regularization Through Flags (`tv_weight`, `contract_w`, `weight_decay`)

We evaluated the three regularization controls in our implementation: total variation regularization (`tv_weight`), the contractive penalty (`contract_w`), and optimizer weight decay (`weight_decay`).

Dense controller. For both inpainting and denoising, these regularization methods were generally not beneficial for the dense controller in terms of final metrics. In most runs, these terms mainly slowed convergence, while the final PSNR/SSIM/LPIPS did not improve at all. The best dense-controller results on both datasets were obtained with very small values or by disabling these regularizers (setting them to 0).

This relates to the fact that our implementation already contains several forms of implicit/structural regularization (e.g., bounded updates through the rollout design, clamping, masking constraints, and in some settings guard-based rejection). In particular, for denoising, the learning rate was the main factor for stability, and this made additional regularization less impactful.

UNet controller. For the UNet controller, regularization was more useful. In our experiments, small values of `tv_weight` and `contract_w` improved stability and training behavior, and could also improve final quality. By contrast, `weight_decay` did not provide gains and was neutral or slightly detrimental.

Final conclusion. The effect of these flags is architecture-dependent:

- **Dense controller:** regularization mostly acts as damping and slows convergence; best results were typically obtained with minimal regularization.
- **UNet controller:** small `tv_weight` and `contract_w` are useful; `weight_decay` is less helpful.

In all cases, when used, these regularization terms should be small, as large values worsen optimization speed and can limit the achievable reconstruction quality.

5.4.7 Best Configuration Inpainting

After the takeaways in the previous section, our best configuration for inpainting removes a lot of the components that were analysed individually and deemed redundant or were even hurting performance. The final architecture used is visible in Figure 5.10.

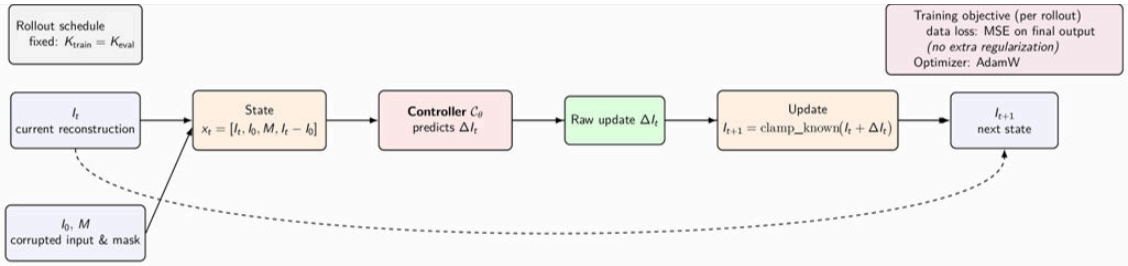


Figure 5.10: Final inpainting step and training pipeline (fixed rollout; no gate/β; no guard/backtracking; optional `corr_clip` as a pure per-step limiter).

5.4.7.1 CIFAR-10 dataset

For CIFAR-10 (32×32 resolution), the best-performing configuration is obtained using the dense controller with fixed rollout depth and no additional regularization. Unlike

our earlier experiments with conservative step sizes and strong clipping, the final configuration removes clipping entirely and aligns training and evaluation dynamics (i.e., $K_{\text{train}} = K_{\text{eval}} = 10$ with fixed rollout).

The final reconstruction metrics are:

- Final PSNR: **28.79 dB**
- Final SSIM: **0.9164**
- Final LPIPS: **0.0065**

These results confirm that once the update magnitude is properly calibrated (by removing overly conservative clipping and unnecessary regularization), the dense controller alone is sufficient to achieve strong iterative refinement. The performance gain is therefore not due to architectural complexity, but due to correctly tuned iterative dynamics.

The exact command used for the best CIFAR configuration can be found in Appendix A.4. The corresponding step-wise PSNR, LPIPS, and SSIM curves are shown in Figures A.1, A.2, and A.3, respectively.

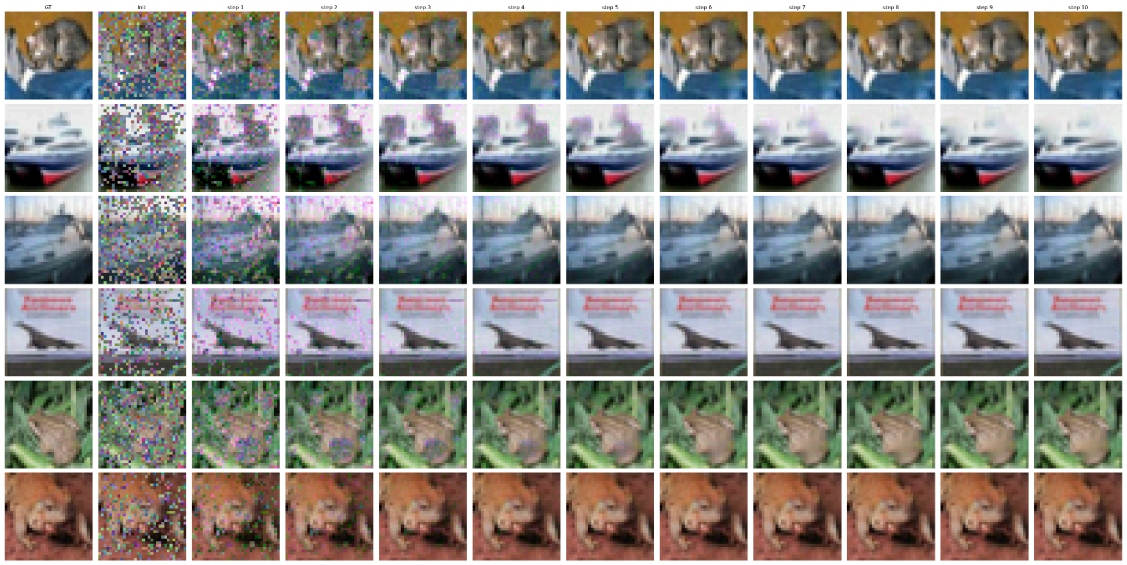


Figure 5.11: Best CIFAR-10 inpainting configuration results

5.4.7.2 CelebA-HQ dataset

For CelebA-HQ, experiments were conducted at 128×128 resolution using the same dense controller architecture and identical training principles as in CIFAR: fixed rollout, no clipping, and no explicit regularization terms.

The final reconstruction metrics are:

- Final PSNR: **30.63 dB**
- Final SSIM: **0.9509**
- Final LPIPS: **0.0233**

Compared to CIFAR-10, the higher-resolution CelebA-HQ dataset shows a similar qualitative behavior: large improvements in early steps, followed by progressively smaller refinements. The convergence is smooth and monotonic, without oscillations, indicating stable iterative dynamics even at higher spatial complexity.

Importantly, these results are achieved without pyramid rollouts, without clipping, and without auxiliary regularization losses. This reinforces one of the central findings of this work: once the update magnitude and rollout alignment are properly tuned, the core iterative mechanism is sufficient to achieve high-fidelity inpainting. The exact command used for the best CelebA-HQ configuration can be found in Appendix A.5. The corresponding step-wise PSNR, LPIPS, and SSIM curves are shown in Figures A.4, A.5, and A.6, respectively.



Figure 5.12: Best CELEBAHQ inpainting configuration results

5.4.8 Best Configuration Denoising

Similarly to the inpainting task, the architecture for denoising also removes the same redundant components. The main difference is that it keeps two regularization components, The final architecture used is visible in Figure 5.13.

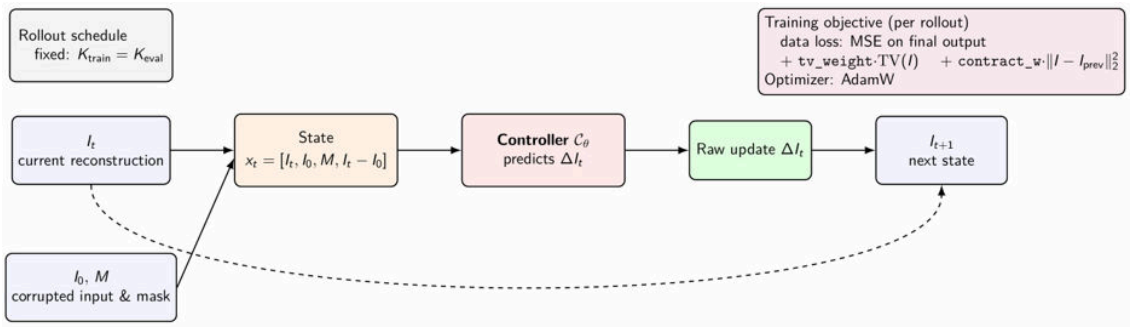


Figure 5.13: Final denoising step and training pipeline (fixed rollout; no gate/β; no guard/backtracking; use of regularization components).

5.4.8.1 CIFAR-10 dataset

For CIFAR-10 (32×32 resolution), the best-performing denoising configuration uses the UNet controller with additive Gaussian noise and fixed rollout depth. Unlike the inpainting case, where the dense controller was sufficient, denoising benefits from stronger spatial modeling capacity. The optimal setup uses $K_{\text{train}} = K_{\text{eval}} = 5$ with fixed rollout and no clipping or explicit regularization.

The final reconstruction metrics are:

- Final PSNR: **26.40 dB**
- Final SSIM: **0.8150**
- Final LPIPS: **0.0200**

The improvement is front-loaded: most of the denoising effect occurs within the first few iterations, with diminishing gains after step 4–5. This suggests that, for small-resolution natural images corrupted with additive Gaussian noise, a limited number of iterative refinements is sufficient when the controller has adequate spatial capacity.

The exact command used for the best CIFAR denoising configuration can be found in Appendix A.6. The corresponding step-wise PSNR, LPIPS, and SSIM curves are provided in Figures A.7, A.8, and A.9, respectively.

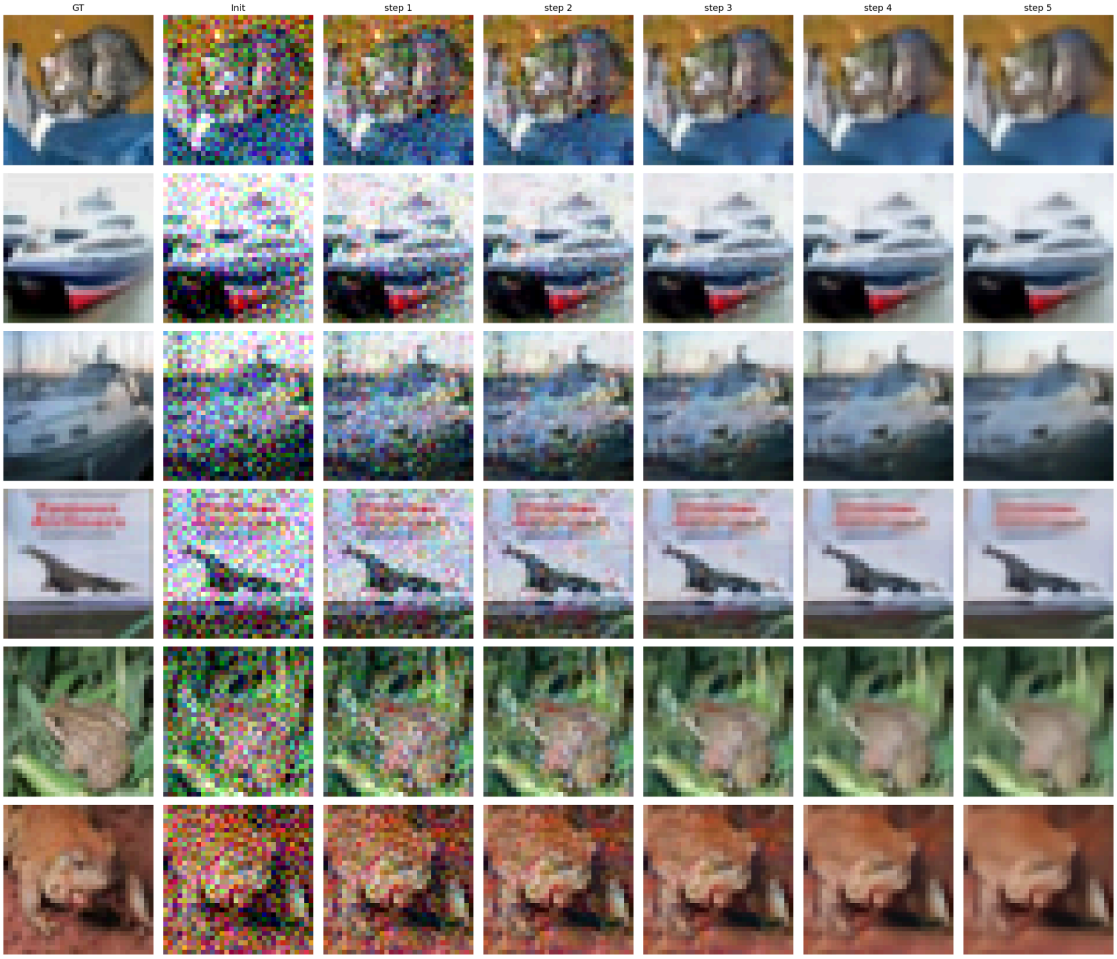


Figure 5.14: Best CIFAR denoising configuration results

5.4.8.2 CelebA-HQ dataset

For CelebA-HQ (128×128 resolution), the UNet controller again provides the best denoising performance under additive Gaussian noise. In contrast to CIFAR-10, the higher-resolution images benefit from a larger rollout depth, with $K_{\text{train}} = K_{\text{eval}} = 10$ yielding the strongest results.

The final reconstruction metrics are:

- Final PSNR: **30.84 dB**
- Final SSIM: **0.8682**
- Final LPIPS: **0.0679**

Compared to CIFAR-10, the denoising process on CelebA-HQ exhibits a longer and more gradual refinement trajectory. The PSNR improvements remain significant up to later iterations, reflecting the increased structural complexity of higher-resolution images.

Importantly, as in the inpainting experiments, these results are achieved without clipping, without pyramid rollouts, and without auxiliary regularization terms. The stability

of the iterative process indicates that, when training and evaluation depths are aligned and update magnitudes are properly calibrated, the NFTM framework extends naturally from inpainting to additive denoising.

The exact command used for the best CelebA-HQ denoising configuration can be found in Appendix A.7. The corresponding step-wise PSNR, LPIPS, and SSIM curves are provided in Figures A.10, A.11, and A.12, respectively.



Figure 5.15: Best CELEB denoising configuration results

5.4.9 Controlling the Task Behavior Through Flags

Our implementation supports both inpainting and denoising using the same code path; the task is selected entirely through the masking/corruption flags. The task is determined by how the binary mask M is generated (which pixels are treated as "unknown"):

- **Inpainting.** We sample a mask with a non-trivial missing region and (optionally) structured occlusions. In practice, we use

  $$\texttt{-pmin} = 0.25, \quad \texttt{-pmax} = 0.5, \quad \texttt{-block_prob} = 0.5,$$

  which produces mixed random missing pixels plus square occlusion blocks. The controller then iteratively reconstructs the unknown region while the known pixels are always enforced with `clamp_known`.

- **Denoising.** We disable masking by marking all pixels as unknown (i.e., no known measurements are enforced). This is done by setting

  $$\texttt{-pmin} = 1, \quad \texttt{-pmax} = 1, \quad \texttt{-block_prob} = 0,$$

  so the mask contains no fixed pixels and the update rule applies everywhere. In our experiments, we typically use this with Gaussian corruption (default) and, when enabled through another flag, additive corruption with `-gaussian_additive`.

Therefore, switching between inpainting and denoising does not require changing the architecture or training loop; it only requires changing `-pmin/-pmax` and `-block_prob` to define whether the algorithm operates under partial observations (inpainting) or fully unobserved/noisy inputs (denoising).

5.5 Main Findings from Experiments

5.5.1 Differences between baselines - Inpainting

The tables compare classical baselines (UNet and TV-L1), the default NFTM setup, and the proposed controller-based variants across CIFAR-10 and CelebA-HQ. Performance is evaluated using both pixel-wise fidelity metrics (PSNR, SSIM) and perceptual quality metrics (LPIPS, FID, KID).

Method	psnr↑	ssim↑	lpips↓	fid↓	kid↓
unet	26.786	0.8807	0.0113	17.776	0.0104
tvl1	25.576	0.8770	0.0124	20.021	0.0119
tinyunet_diffusion	21.81	0.715	0.0249	119.06	0.1179
bigunet_diffusion	23.27	0.786	0.0173	67.20	0.0628
nftm_default	21.494	0.6989	0.0272	90.014	0.0837
previous_dense_controller	28.54	0.914	0.00644	6.693	0.0028
previous_unet_controller	28.019	0.906	0.0075	8.070	0.00349
best_config_dense_controller	**28.79**	**0.9164**	**0.0065**	**6.381**	**0.0025**

Table 5.8: Baseline comparison on CIFAR-10 inpainting. Midterm results start with the word "previous_".

Method	psnr↑	ssim↑	lpips↓	fid↓	kid↓
unet	27.272	0.8685	0.0434	51.156	0.0550
tvl1	26.688	0.9074	0.0335	27.126	0.0244
tinyunet_diffusion	20.12	0.589	0.3091	121.58	0.1476
bigunet_diffusion	25.11	0.860	0.0685	68.27	0.0808
default_dense	20.253	0.5929	0.2077	98.335	0.1079
previous_dense_controller	29.342	0.932	0.01525	7.5706	0.00624
previous_unet_controller	30.002	0.9389	**0.0151**	9.589	0.0085
best_conf_dense_controller	**30.63**	**0.9509**	0.0233	**7.493**	**0.00612**

Table 5.9: Baseline comparison on CelebA-HQ inpainting. Midterm results start with the word "previous_".

5.5.2 Summary of Best Achieved Results

For clarity, Table 5.10 summarizes the best configurations obtained for both inpainting and denoising across CIFAR-10 and CelebA-HQ, reporting the final reconstruction quality and the resolution used.

These results correspond to the fully tuned iterative setups described in previous sections, where training and evaluation rollouts are aligned and update magnitudes are properly calibrated.

Dataset	Task	Resolution	PSNR↑	SSIM↑	LPIPS↓
CIFAR-10	Inpainting	32×32	28.79	0.9164	0.0065
CelebA-HQ	Inpainting	128×128	30.63	0.9509	0.0233
CIFAR-10	Denoising	32×32	26.40	0.8150	0.0200
CelebA-HQ	Denoising	128×128	30.84	0.8682	0.0679

Table 5.10: Best achieved results for inpainting and denoising across datasets and resolutions.

Chapter 6

Conclusions and Future Work

6.1 Conclusions

This project studied NFTM-style iterative reconstruction for two related inverse problems: *masked* restoration (image inpainting) and *additive / real-noise* restoration (image denoising). Rather than treating restoration as a single forward mapping, we framed both tasks as a learned dynamical system: starting from a corrupted initialization, a compact controller is applied repeatedly to predict corrections that progressively refine the image estimate.

Main takeaway. Across our experiments, the dominant driver of performance is the **iterative correction mechanism itself**, not architectural complexity. When the update dynamics are properly calibrated (step magnitude and rollout alignment), even small controllers achieve strong results. Conversely, early underperformance was mainly caused by **overly conservative update settings** (e.g., step-size/regularization choices that made each iteration too small), not by insufficient model capacity.

Key findings.

- **Iterative NFTM controllers outperform baselines.** On both CIFAR-10 and CelebA-HQ inpainting, the iterative controllers (dense and U-Net) consistently outperform a one-shot Tiny U-Net baseline and a classical TV–L1 variational baseline across fidelity (PSNR/SSIM) and perceptual/distributional metrics (LPIPS/-FID/KID).
- **Performance is driven by learned updates over time.** The gains come from repeatedly re-evaluating and correcting the current reconstruction, rather than from a more expressive single-pass generator.
- **Tuning the dynamics matters more than adding mechanisms.** In practice, the most reliable improvements came from (i) removing unnecessary conservatism in the update magnitude and (ii) matching training and evaluation rollout behavior (e.g., fixed depth, $K_{\text{train}} \approx K_{\text{eval}}$), not from adding extra stabilization modules.
- **Generalization beyond inpainting is feasible.** The same iterative formulation transfers naturally to denoising: we verified the approach on synthetic additive Gaussian noise and extended it to **real camera noise** using SIDD via a tile-based training and reconstruction pipeline, showing stable step-wise improvement and visually coherent stitched outputs.

6.2 Limitations

Despite strong empirical results, several limitations remain:

- **Hyperparameter sensitivity.** NFTM performance depends strongly on update-dynamics parameters (rollout depth, controller choice, learning rate etc.). Small mis-tuning can slow convergence or destabilize long rollouts.
- **Compute cost vs. one-shot models.** Iterative refinement requires multiple controller evaluations per image. Even with small controllers, inference can be slower than single-pass baselines, which matters for deployment or high-resolution restoration.
- **Stability mechanisms do not replace stability-aware training.** Per-step acceptance/rejection (guard-style checks) can prevent some local failures, but it does not guarantee that the learned update map remains stable under optimization. Avoiding unstable regimes still requires appropriate training choices (notably learning-rate and step-magnitude control).

6.3 Future Work

The most valuable next steps are those that directly address robustness, generalization, and competitiveness:

- **Broader inverse problems.** The same iterative controller loop can be reused across many restoration operators: each step proposes a correction, and the rollout progressively enforces consistency with the observation while adding missing structure. Concretely:
  - **Super-resolution:** treat low resolution as missing high-frequency content; each iteration adds detail while remaining consistent with the low-res input.
  - **Deblurring / deconvolution:** model blur as a known forward operator and iteratively sharpen the estimate under a data-consistency constraint.
  - **Dehazing / deraining / desnowing:** corruption is structured rather than random; the same refine-and-correct updates can progressively remove artifacts without destroying edges.
  - **Compression artifact removal:** iteratively suppress blocking/ringing while preserving textures and high-contrast boundaries.
  - **Colorization / demosaicing:** treat missing color channels or CFA sampling as partial observation; refinements fill in plausible color conditioned on the current reconstruction.
  - **Inpainting variants:** extend masked reconstruction to harder masks (irregular holes, text/object removal) without changing the rollout principle.
  - **Blind restoration:** jointly infer the degradation (e.g., noise level, blur kernel) and the clean image within the same iterative loop, aiming to restore without knowing the corruption a priori.

  - **Beyond images:** the framework is not image-specific; it can extend to inverse problems such as audio denoising, MRI/CT reconstruction, microscopy deconvolution, and time-series imputation.

- **Richer and more realistic noise models.** Extend denoising experiments beyond additive Gaussian and SIDD to additional noise types (e.g., speckle, salt-and-pepper) and more datasets, emphasizing cross-dataset generalization (train on one, test on another).

- **Modern baselines and stronger comparisons.** Compare against more contemporary inpainting/denoising baselines (e.g. deep equilibrium models) under matched compute budgets to position NFTM fairly. In addition, extend and re-run the current baseline suite (one-shot U-Net, TV-based methods, etc.) in a denoising setting so comparisons remain consistent across both tasks.

- **Efficiency work.** Reduce inference cost via early stopping criteria, dynamic step allocation, or distillation of multi-step behavior into fewer steps while preserving the iterative refinement benefits.

Appendix A

Appendix

This appendix reports the exact command-line parameters used for all baseline and NFTM experiments.

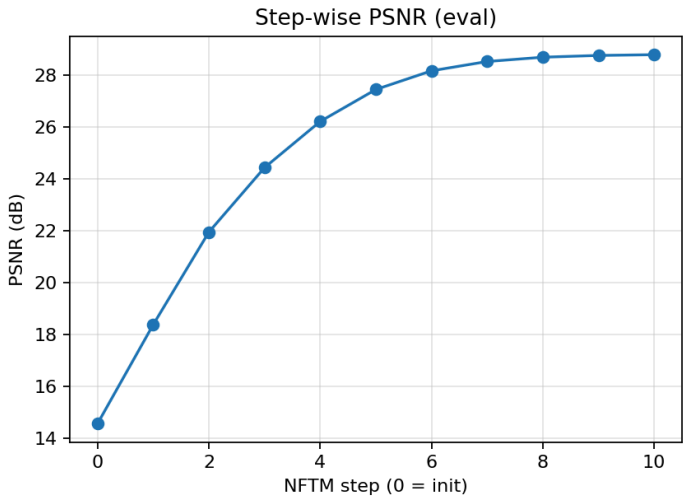


Figure A.1: PSNR curve for the best configuration of the inpainting of CIFAR dataset

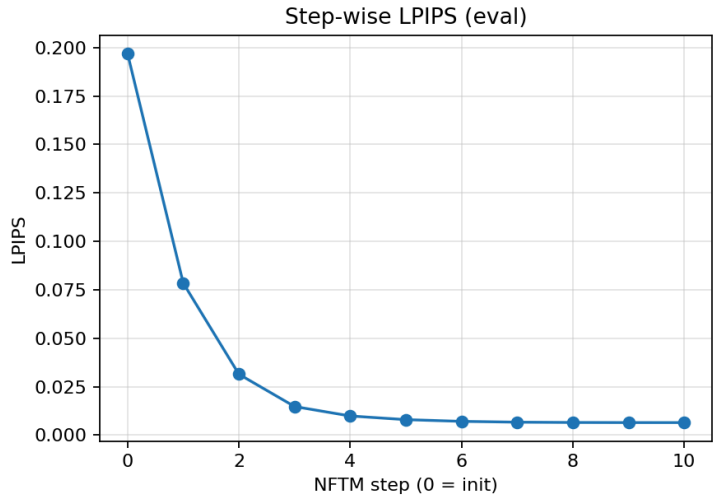


Figure A.2: LPIPS curve for the best configuration of the inpainting of CIFAR dataset

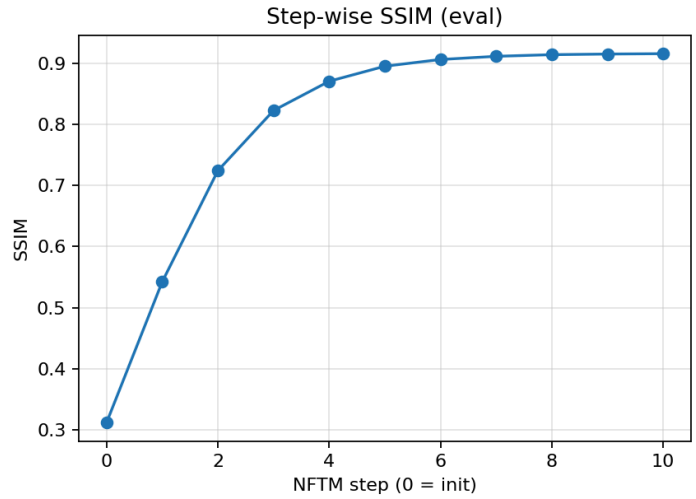


Figure A.3: SSIM curve for the best configuration of the inpainting of CIFAR dataset

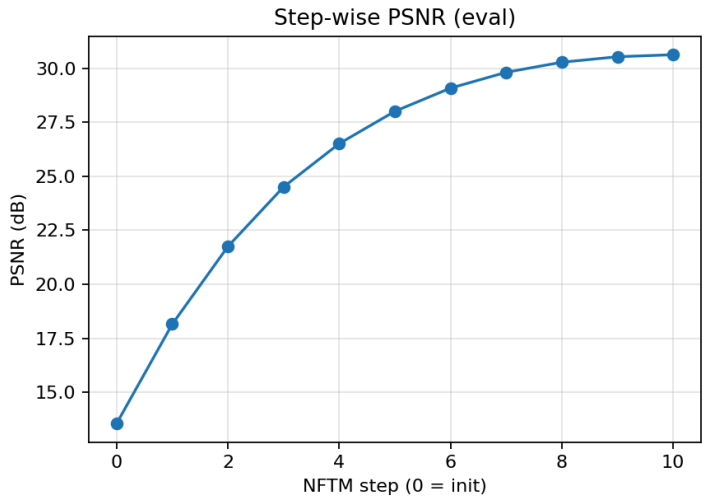


Figure A.4: PSNR curve for the best configuration of the inpainting of Celeb dataset

Category	Flag	Description (default)
Data / I/O	`-train_dataset`	Training dataset: `{cifar, celebahq}` (default: `cifar`).
	`-benchmark`	Evaluation dataset / benchmark: `{cifar, set12, cbsd68, celebahq}` (default: `cifar`).
	`-img_size`	Working resolution, resized if needed: `{32, 64, 128, 256}` (default: 32).
	`-save_dir`	Output directory for plots, logs and metrics (default: `out`).
	`-save_metrics`	Also write `metrics.json` and `*_curve.npy` to `save_dir`.
	`-save_epoch_progress`	Save per–epoch step grids (and GIF if available) for the first eval batch.
	`-viz_scale`	Upsampling factor for PNG/GIF visualisations (default: 1.0, i.e. native).
	`-use_wandb`	Stream training and eval metrics to Weights & Biases (default: off).
	`-seed`	Global RNG seed for reproducibility (default: 0).
	`-device`	Compute device (default: `cuda` if available, else `cpu`).
Model	`-controller`	Controller architecture: `dense` or `unet` (default: `dense`).
	`-width`	Channel width for the dense controller (default: 48).
	`-unet_base`	Base channels for Tiny U-Net controller (default: 10).
	`-gate`	Enable controller gate: $I \leftarrow I + \beta\,(g \odot \Delta I)$ (default: off; gate ignored).
Optimisation	`-epochs`	Number of training epochs (default: 10).
	`-batch_size`	Mini-batch size (default: 128).
	`-lr`	Learning rate for AdamW (default: 2×10^{-3}).
	`-weight_decay`	Weight decay for AdamW (default: 10^{-4}).
	`-tv_weight`	Weight of the TV-L1 smoothness penalty (default: 0.01).
	`-contract_w`	Weight of the contractive penalty (default: 10^{-3}).
Rollout & step sizes	`-K_train`	Target rollout depth during training (default: 8).
	`-K_eval`	Rollout depth during evaluation / visualisation (default: 12).
	`-fixed_K`	Use fixed rollout length per batch ($K_{\text{curr}} = K_{\text{train}}$); disables random rollout-length sampling (default: off).
	`-no_curriculum`	Disable rollout curriculum; use $K_{\text{train}} = K_{\text{target}}$ from epoch 1 (default: off).
	`-beta_start`	Initial damping factor β (default: `None`, i.e. β disabled).
	`-beta_max`	Optional cap on β during training (default: `None`).
	`-beta_anneal`	Optional per–epoch increment of β (default: `None`).
	`-beta_eval_bonus`	Optional extra β added only at evaluation (default: `None`).
	`-corr_clip`	Base per–step correction clipping threshold; setting to 0 disables clipping (default: 0.1).
	`-guard_in_train`	Enable energy-based guard during training (default: off).
	`-step_loss`	Data-loss accumulation: `final` (only last step) vs. `linear` (linearly weighted over steps) (default: `final`).
Masking / corruption	`-pmin`, `-pmax`	Range of missing-pixel fractions for random masks (default: 0.25–0.5).
	`-block_prob`	Probability of adding random square occlusion blocks (default: 0.5).
	`-noise_std`	Std. of Gaussian corruption noise for unknown pixels (default: 0.3).
	`-gaussian_additive`	Use additive Gaussian corruption inside the unknown region (default: off).
	`-eval_noise_sweep`	Run evaluation across multiple corruption types and save per-noise results (default: off).
Pyramid rollout	`-pyramid`	Comma-separated list of resolutions for coarse-to-fine rollout (e.g. `16,32,64`); empty means single-scale (default: empty).
	`-pyr_steps`	Optional steps per pyramid level (must sum to `K_eval`); empty means auto split (default: empty).
Jacobian logging	`-log_jacobian`	Estimate and log a sampled $\sigma_{\max}(J)$ of the update map each epoch (default: off).
	`-jacobian_K`	K-step map for Jacobian probing: 1=single-step, $>$ 1=K-step composed map (default: 1).
	`-jacobian_batches`	Number of mini-batches to average for Jacobian estimate (default: 1).
	`-jacobian_power_iters`	Power-iteration steps for $\sigma_{\max}$ estimate (default: 6).

Table A.1: Command–line flags of the `image_inpainting.py` script (current version).

Method	Entry point	Exact parameters
unet	`train_unet.py`	`python train_unet.py --epochs 20 --batch_size 256 --lr 0.002 --weight_decay 0.0001 --tv_weight 0.01 --seed 0 --base 10 --target_params 46375 --num_workers 2 --device cuda --save_dir out_cifar32/unet`
tvl1	`baselines.inpainting.tvl1_baseline`	`python baselines.inpainting.tvl1_baseline --iters 250 --lam 80.0 --tvw 0.1 --tau_p 0.25 --tau_d 0.25 --batch_size 256 --num_workers 2 --seed 0 --save_dir out_cifar32/tvl1`
nftm_default	`image_inpainting.py`	`python image_inpainting.py --train_dataset cifar --benchmark cifar --img_size 32 --epochs 20 --save_metrics --save_dir out_cifar32/nftm`
dense_controller	`image_inpainting.py`	`python image_inpainting.py --controller dense --beta_eval_bonus 0.1079 --beta_max 0.8800 --beta_start 0.4230 --contract_w 0.0005 --corr_clip 0.3276 --epochs 30 --img_size 32 --K_eval 25 --K_train 22 --tv_weight 0.0050 --seed 0 --device cuda --save_metrics --save_dir runs/inpainting/nftm_dense`
unet_controller	`image_inpainting.py`	`python image_inpainting.py --controller unet ----beta_eval_bonus 0.0948 --beta_max 0.7829 --beta_start 0.4486 --contract_w 0.0005 --corr_clip 0.2823 --epochs 50 --img_size 32 --K_eval 16 --K_train 24 --tv_weight 0.0046 --seed 0 --device cuda --save_metrics --save_dir runs/inpainting/nftm_unet`
nftm_pyramid	`image_inpainting.py`	`python image_inpainting.py --controller dense --epochs 30 --K_train 20 --K_eval 30 --seed 0 --device cuda --save_metrics --train_dataset cifar --benchmark cifar --img_size 32 --pyramid 16,32 --pyr_steps 3,27 --save_dir runs/inpainting/nftm_dense`

Table A.2: Exact parameters used for CIFAR-10 baseline and NFTM comparisons.

Method	Entry point	Exact parameters
unet	`train_unet.py`	`python train_unet.py --epochs 20 --batch_size 256 --lr 0.002 --weight_decay 0.0001 --tv_weight 0.01`
tvl1	`tvl1_baseline`	`python tvl1_baseline --iters 250 --lam 80.0 --tvw 0.1 --tau_p 0.25 --tau_d 0.25`
nftm_20_epochs	`image_inpainting.py`	`python image_inpainting.py --train_dataset celebahq --epoch 20`
dense_controller_30_epochs	`python image_inpainting.py`	`python image_inpainting.py --controller dense --seed 0 --device cuda --save_metrics --beta_eval_bonus 0.1101 --beta_max 0.8965 --beta_start 0.4428 --contract_w 0.0004 --corr_clip 0.2353 --epochs 30 --K_eval 20 --K_train 20 --tv_weight 0.0041`
unet_controller_50_epochs	`image_inpainting.py`	`python image_inpainting.py --controller unet --epochs 50 --K_train 17 --K_eval 23 --beta_eval_bonus 0.077 --beta_max 0.8722 --beta_start 0.5784 --contract_w 0.0001 --corr_clip 0.3257 --tv_weight 0.0020 --seed 0 --device cuda --save_metrics`
nftm_pyramid_30_epochs	`image_inpainting.py`	`python image_inpainting.py --controller dense --epochs 30 --K_train 20 --K_eval 30 --seed 0 --device cuda --save_metrics --pyramid 16,32,64 --pyr_steps 3,10,17`
nftm_pyramid_10_epochs	`image_inpainting.py`	`python image_inpainting.py --epochs 10 --batch_size 128 --K_train 10 --K_eval 13 --beta_start 0.28 --beta_max 0.6 --beta_anneal 0.03 --beta_eval_bonus 0.05 --corr_clip 0.1 --tv_weight 0.01 --pmin 0.25 --pmax 0.5 --block_prob 0.5 --noise_std 0.3 --width 48 --guard_in_train --save_metrics --pyramid 16,32,64 --pyr_steps 1,2,10`

Table A.3: Exact parameters used for CelebA-HQ baseline and NFTM comparisons.

Method	Entry point	Exact parameters
dense_controller_best_cifar32	`image_inpainting.py`	`python image_inpainting.py --train_dataset cifar --benchmark cifar --img_size 32 --controller dense --width 48 --epochs 30 --batch_size 128 --K_train 10 --K_eval 10 --pmin 0.25 --pmax 0.5 --block_prob 0.5 --corr_clip 0 --noise_std 0.3 --tv_weight 0 --contract_w 0 --weight_decay 0 --no_curriculum --fixed_K`

Table A.4: Exact parameters used for the best CIFAR-10 inpainting configuration (32×32).

Method	Entry point	Exact parameters
dense_controller_best_celeb128	`image_inpainting.py`	`python image_inpainting.py --train_dataset celebahq --benchmark celebahq --img_size 128 --controller dense --width 48 --epochs 20 --batch_size 64 --K_train 10 --K_eval 10 --pmin 0.25 --pmax 0.5 --block_prob 0.5 --corr_clip 0 --noise_std 0.3 --tv_weight 0 --contract_w 0 --weight_decay 0 --no_curriculum --fixed_K`

Table A.5: Exact parameters used for the best CelebA-HQ inpainting configuration (128×128).

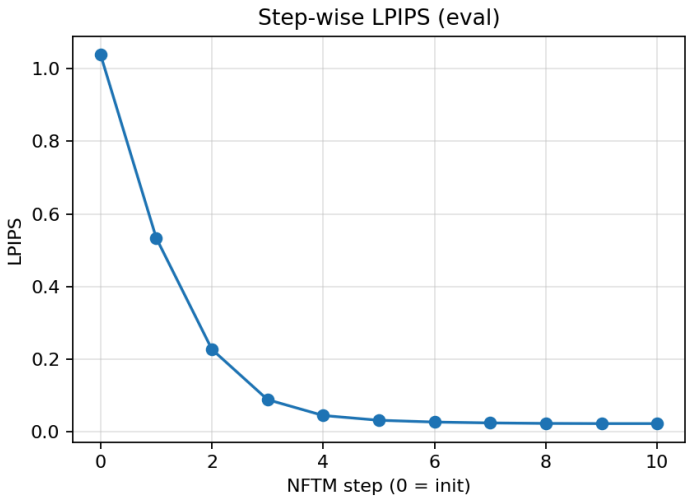


Figure A.5: LPIPS curve for the best configuration of the inpainting of Celeb dataset

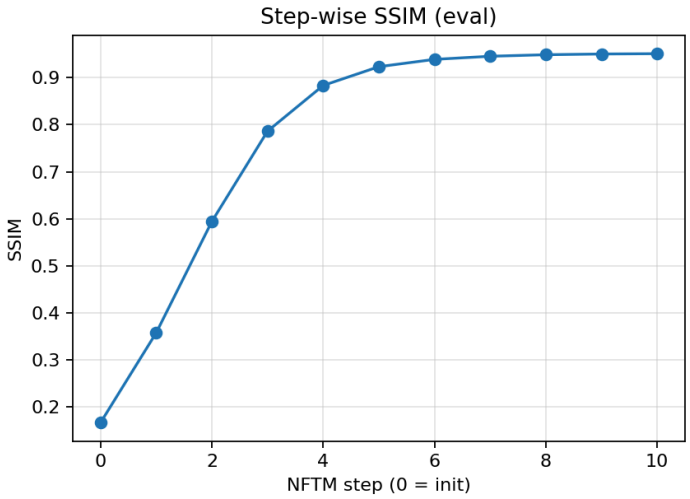


Figure A.6: SSIM curve for the best configuration of the inpainting of Celeb dataset

Method	**Entry point**	**Exact parameters**
unet_controller_best_cifar32_denoise	`image_inpainting.py`	`python image_inpainting.py --train_dataset cifar --benchmark cifar --img_size 32 --lr 3e-3 --controller unet --width 48 --epochs 30 --batch_size 128 --K_train 5 --K_eval 5 --pmin 1 --pmax 1 --block_prob 0 --noise_std 0.3 --corr_clip 0 --weight_decay 0 --no_curriculum --fixed_K --gaussian_additive`

Table A.6: Exact parameters used for the best CIFAR-10 denoising configuration (32×32, additive Gaussian noise).

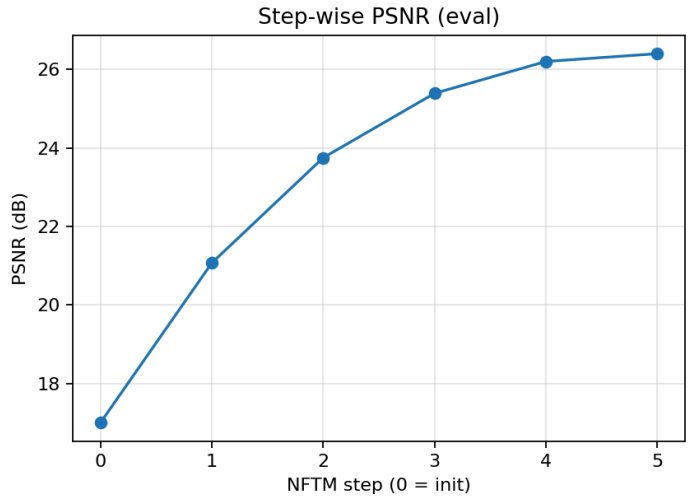


Figure A.7: PSNR curve for the best configuration of the denoising of CIFAR dataset

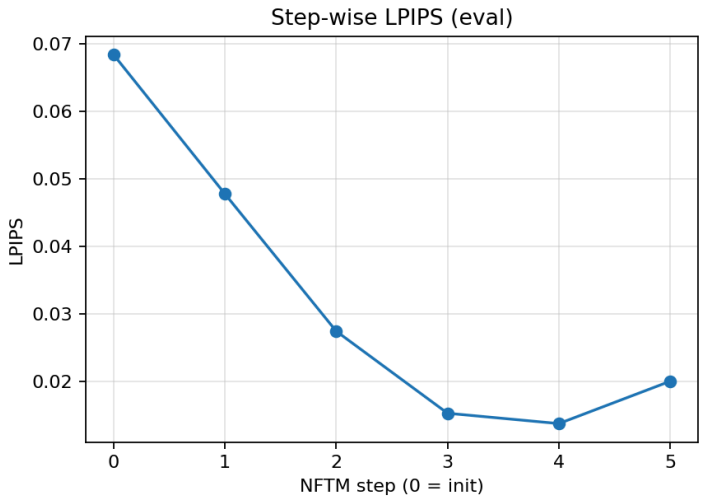


Figure A.8: LPIPS curve for the best configuration of the denoising of CIFAR dataset

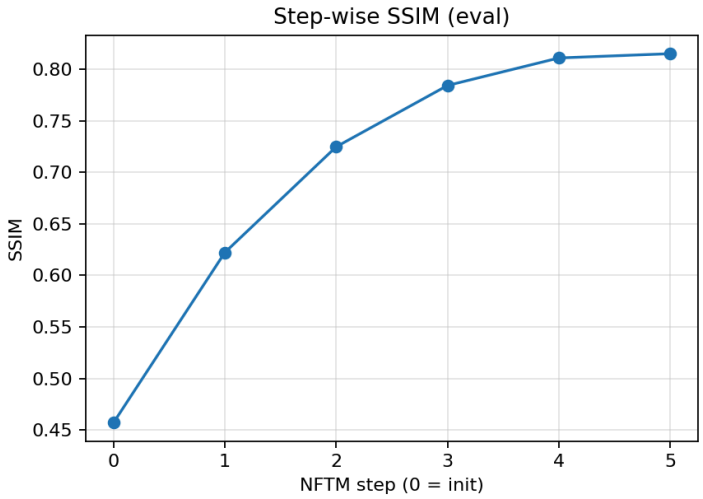


Figure A.9: SSIM curve for the best configuration of the denoising of CIFAR dataset

Method	Entry point	Exact parameters
unet_controller_best_celeb128_denoise	`image_inpainting.py`	`python image_inpainting.py --train_dataset celebahq --benchmark celebahq --img_size 128 --lr 3e-3 --controller unet --width 48 --epochs 20 --batch_size 64 --K_train 10 --K_eval 10 --pmin 1 --pmax 1 --block_prob 0 --noise_std 0.3 --corr_clip 0 --weight_decay 0 --no_curriculum --fixed_K --gaussian_additive`

Table A.7: Exact parameters used for the best CelebA-HQ denoising configuration (128×128, additive Gaussian noise).

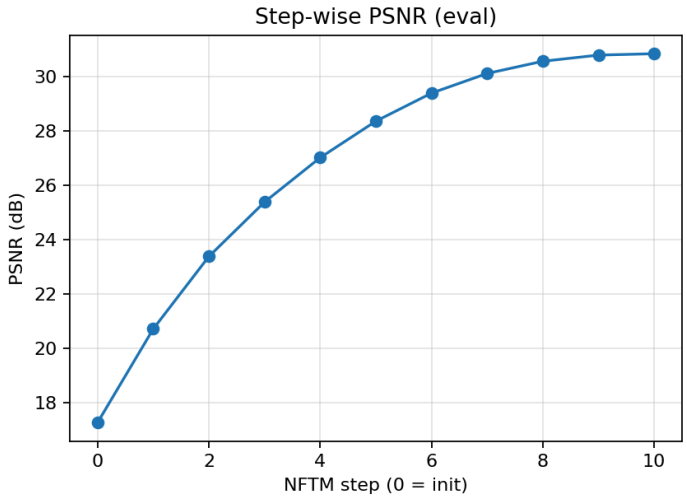


Figure A.10: PSNR curve for the best configuration of the denoising of Celeb dataset

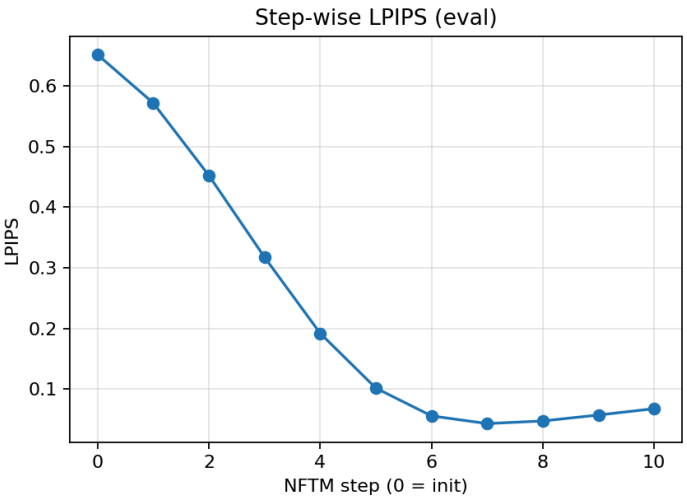


Figure A.11: LPIPS curve for the best configuration of the denoising of Celeb dataset

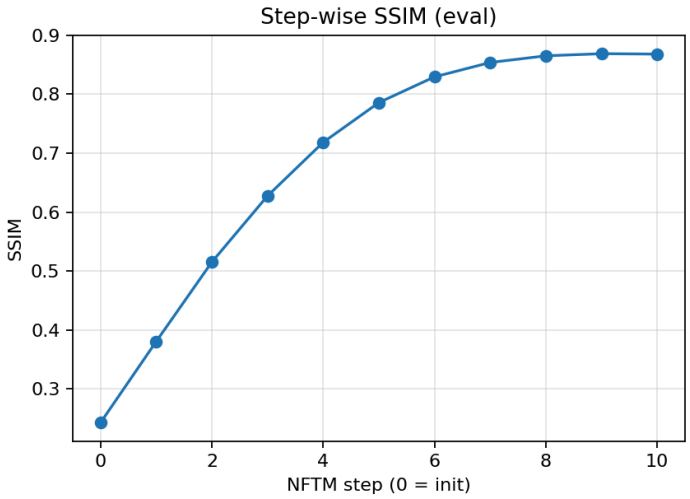


Figure A.12: SSIM curve for the best configuration of the denoising of Celeb dataset

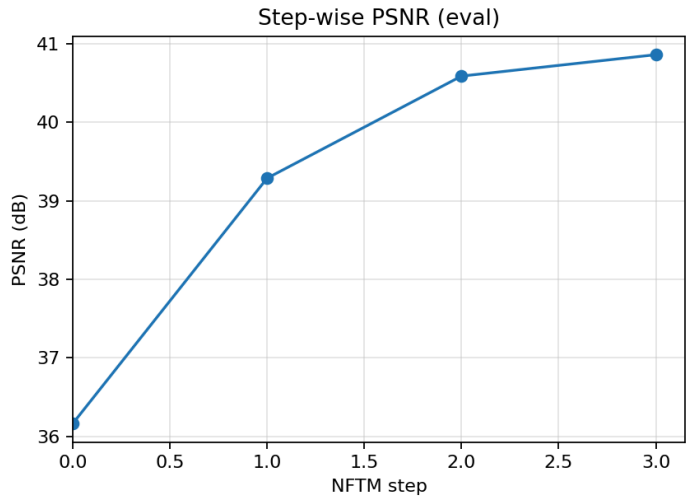


Figure A.13: PSNR curve for the best configuration of real camera noise denoising

Method	Entry point	Exact parameters
nftm_best_sidd_denoise_overlap_guard	sidd_denoising.py	python sidd_denoising.py --train_dataset sidd --benchmark sidd --sidd_root /gpfs/workdir/balazsk/NFTM/benchmarks/SIDD/SIDD_Medium_Srgb --patch_size 64 --patch_stride 32 --epochs 12 --batch_size 48 --K_train 10 --K_eval 3 --beta_start 0.18 --beta_max 0.60 --beta_anneal 0.03 --beta_eval_bonus 0.05 --corr_clip 0.09 --contract_w 0.0008 --tv_weight 0.00002 --guard_in_train --gate --step_loss linear --eval_every 1 --eval_max_batches 120 --save_epoch_progress --sidd_viz_samples 100 --sidd_viz_pngs 10 --sidd_viz_max_steps 8 --sidd_limit_tiles 0 --sidd_epoch_tiles 8000 --save_metrics --save_dir out/sidd_denoise_test13_overlap_guard --sidd_save_full_images --sidd_full_max_images 30 --sidd_full_batch 24 --sidd_full_tile_pad 16 --seed 13

Table A.8: Exact parameters used for the best SIDD denoising configuration (64×64 overlapping patches, guarded NFTM rollout with linear step-weighted loss).

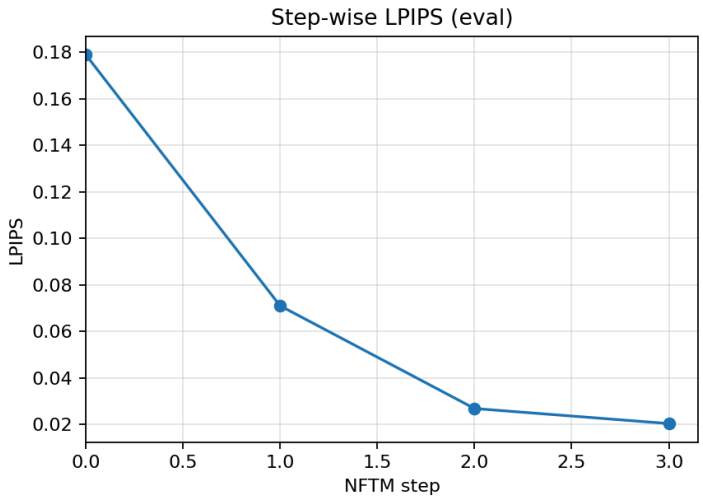


Figure A.14: LPIPS curve for the best configuration of real camera noise denoising

Bibliography

[1] Inpainting example figure (researchgate). https://www.researchgate.net/figure/Inpainting-example-1-obtained-by-applying-the-proposed-method-to-larger-missing-areas_fig11_269588939. Accessed: 2026-02-27. (Cited on page 1.)

[2] Vincent Mazel. Image denoising. https://vincmazet.github.io/bip/restoration/denoising.html. Accessed: 2026-02-27. (Cited on page 2.)

[3] Akash Malhotra and Nacéra Seghouani. Neural field turing machine: A differentiable spatial computer. 2025. (Cited on page 3.)

[4] Rafael C. Gonzalez and Richard E. Woods. Digital Image Processing. Prentice Hall, Upper Saddle River, NJ, USA, 3 edition, 2008. (Cited on page 9.)

[5] Zhou Wang, Alan C Bovik, Hamid R Sheikh, and Eero P Simoncelli. Image quality assessment: from error visibility to structural similarity. IEEE transactions on image processing, 13(4):600–612, 2004. (Cited on page 9.)

[6] Richard Zhang, Phillip Isola, Alexei A Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In Proceedings of the IEEE conference on computer vision and pattern recognition, pages 586–595, 2018. (Cited on page 10.)

[7] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. Gans trained by a two time-scale update rule converge to a local nash equilibrium. Advances in neural information processing systems, 30, 2017. (Cited on page 10.)

[8] Mikołaj Bińkowski, Danica J Sutherland, Michael Arbel, and Arthur Gretton. Demystifying mmd gans. arXiv preprint arXiv:1801.01401, 2018. (Cited on page 11.)

[9] Jianhong Shen and Tony F Chan. Mathematical models for local nontexture inpaintings. SIAM Journal on Applied Mathematics, 62(3):1019–1043, 2002. (Cited on page 12.)

[10] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015, volume 9351 of Lecture Notes in Computer Science, pages 234–241. Springer, Cham, 2015. (Cited on page 12.)

[11] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In Advances in Neural Information Processing Systems, volume 33, pages 6840–6851, 2020. (Cited on pages 13 and 19.)

[12] Andreas Lugmayr, Martin Danelljan, Andres Romero, Fisher Yu, Radu Timofte, and Luc Van Gool. Repaint: Inpainting using denoising diffusion probabilistic models. In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pages 11461–11471, 2022. (Cited on pages 13, 19 and 20.)

[13] Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical Report TR-2009, University of Toronto, 2009. (Cited on page 14.)

[14] Tero Karras, Timo Aila, Samuli Laine, and Jaakko Lehtinen. Progressive growing of gans for improved quality, stability, and variation. arXiv preprint arXiv:1710.10196, 2017. (Cited on page 14.)

[15] badasstechie. Celeba-hq resized (256x256). https://www.kaggle.com/datasets/badasstechie/celebahq-resized-256x256, 2020. Kaggle Dataset. (Cited on page 15.)

[16] Afshin Abdelhamed, Stephen Lin, and Michael S. Brown. A high-quality denoising dataset for smartphone cameras. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2018. (Cited on page 15.)

[17] Kai Zhang, Wangmeng Zuo, Yunjin Chen, Deyu Meng, and Lei Zhang. Beyond a gaussian denoiser: Residual learning of deep cnn for image denoising. IEEE transactions on image processing, 26(7):3142–3155, 2017. (Cited on page 16.)

[18] Sharon Zhou. How diffusion models work. https://learn.deeplearning.ai/courses/diffusion-models, 2023. DeepLearning.AI online course. (Cited on pages 19 and 21.)

[19] Mésocentre Paris-Saclay. Mésocentre paris-saclay – ruche hpc cluster / user documentation. https://mesocentre.pages.centralesupelec.fr/user_doc/. Accessed: 2025-12-10. (Cited on page 22.)

[20] Shuyi Qu, Zhenxing Niu, Kaizhu Huang, Jianke Zhu, Matan Protter, Gadi Zimerman, and Yinghui Xu. Structure first detail next: Image inpainting with pyramid generator, 2021. (Cited on page 26.)

[21] Leilei Cao, Tong Yang, Yixu Wang, Bo Yan, and Yandong Guo. Generator pyramid for high-resolution image inpainting, 2020. (Cited on page 26.)

[22] Depth Supervision in Deep Networks — emergentmind.com. https://www.emergentmind.com/topics/depth-supervision-ds. [Accessed 11-12-2025]. (Cited on page 28.)